

Dear Meta Reviewer and Referees:

We have substantially revised the paper following the Referees’ suggestions. For the convenience of Referees, changes for addressing the comments of R1, R2 and R3 are color-coded in blue, orange, and red, respectively.

We would like to thank the Referees for insightful comments and for supporting this work!

Below please find our responses to the reviews.

Response to the required changes raised by Meta Reviewer.

(1) Extending the experimental evaluation to include: a) datasets of varying characteristics (R1O3, R2O3) and b) an ablation study to assess the impact of that the quality of the discovered rules has on the overall effectiveness of the approach (R2O1, R3O1)

(2) Extending the explanations to: a) render the paper more self contained and accessible (R1O1, R1O2, R2O4, R3O2, R3O3), b) clarify the offline costs (R2O2).

(3) Introduce the preliminaries about the dual patterns (R1O4) to make the paper self-contained.

(4) Make front size in the tables and figures follow the submission format.

[A] We have revised the paper substantially and addressed all the issues raised: issue (1.a) in the response to R1[O3] and R2[O3&D3]; (1.b) in the response to R2[O1&D1]; (2.a) in the response to R1[O1–O2, Additional remarks], R2[O4&D4] and R3[O2–O3]; (2.b) in the response to R2[O2&D2]; (3) in the response to R1[O4]; and (4) in the response to R1[O1].

Besides, we have addressed other concerns of the Referees. We have (a) clarified the limitations and possible extensions of our method in the response to R2[O4&D4, D5–D6, D9]; (b) discussed the impact of certain factors on soft labeling and data enrichment in the response to R2[D7–D8]; (c) improved the presentations in the response to R3[O1, Additional remarks]; and (d) released the source codes on GitHub.

Response to the comments of Reviewer #1.

[O1] *There is too much material in the paper. Section III.A which introduces the crucial LBRs is too dense to be reader-friendly; it does not seem to reference Fig. 2 at all; there is no example of rules into text, without any examples, makes it very hard to read. Also, Table III is heavily compressed. The authors should decide to take something out of the paper, to the extent of about 1 page, and use that space to properly develop Section III.*

[A] Thanks! To improve the readability, we have substantively expanded Section III-A by adding (1) an example for illustrating dual patterns (Example 2, pp. 4, please also see our response to R1[O4]); and (2) another new example with five LBR rules explained in details (Example 3, pp. 4–5).

To make room for the newly included examples, experiments and discussions, we have simplified the descriptions of

some technical details which are not crucial.

We have also reorganized Table III (pp. 9) and Table V (pp. 11, previous Table IV) to make them more concise.

[O2] *The authors leave out some “data enrichment” delegated to the technical report. It is not clear to the reader of the paper, why are the graphs enriched, and what the impact of that is on the experimental results.*

[A] We have made the following changes to clarify the motivation and practical effectiveness of data enrichment.

(1) In Section IV (pp. 6), we have discussed the need for data enrichment. In fact, a challenge to label imputation is missing information. In practice, the training graph G is often not constrained by a schema and thus, important features for training a GNN $\mathcal{M}$ are sometimes missing from G . Moreover, GNN models may experience an accuracy drop of up to 50% due to 10% of missing node features [1]. To this end, LBoost offers an optional data enrichment module that extracts characteristic features from external data to help the training of $\mathcal{M}$.

(2) In the experimental setting (Section VI, pp. 9–10), we have further clarified that we aligned entities in real-life graphs with external data from OAG [2], [3], Amazon [4] and Wiki-data [5] as knowledge graphs, and chose Mistral-7B [6] as the base LLM for selecting characteristic features (attributes) that are helpful for simplifying classification boundary learning. The module enhances the semantic information of real-life datasets with, e.g., product descriptions, user comments for e-commerce graphs, and user profiles for social networks.

(3) We have included a detailed ablation study in Exp-3 of Section VI (pp. 11–12) to study the impact of data enrichment. As shown in Figure 5(a) for GCN, the accuracy first rises when m (i.e., the number of characteristic features (attributes) added by enrichment) is varied from 0 to 3 or 4, and then decreases very slowly. The highest accuracy is observed at $m = 3$ or 4. This is because that too few features lead to insufficient information and over-fitting for the training, while too many features may introduce redundant data that is irrelevant to classification.

For training time, Figure 5(b) shows that it exhibits a steady yet small increase as m gets larger. Hence LBoost is able to improve the accuracy of GCN while warranting efficient training, by selecting only important features for classification.

[O3] *The authors reached modest improvements (1% to 2%) on real life datasets and much more significant ones (around 15%) on synthetic datasets. The fact that the gains are much weaker on real world data than on synthetic data somehow reduce the excitement wrt the complex process set up by the authors.*

[A] Thanks! We would like to clarify that the previous results were obtained over labeled graphs that have possibly injected noises, but carry no unlabeled nodes. However, in real-world applications, not many nodes are associated with labels.

In light of this, we have conducted a new test to check the effectiveness of LBoost in handling limited labels (Section VI,

pp. 10), a case that is closer to the real situation. Fixing $\rho\% = 0\%$, i.e., no noise is introduced, we masked available labels in the real-life training sets, with a ratio controlled from 0% to 40%. As reported in Table IV, LBoost improves the accuracy of GCN better when the amount of labeled data decreases. On average, it improves the accuracy by 4.0% when 20% available labels are masked. These justify the need for applying LBRs to real graphs with limited labeled data, without injected noises.

[O4] *In Fig. 2, all the dual patterns appear to have identical left- and right-hand parts (up to replacing x with y). However, the text definition of dual patterns does not seem to enforce this equality. This is confusing. The authors should use Fig. 2 properly to illustrate the “easy” and (possible) “harder”/less trivial cases of dual patterns, and comment in it within the text.*

[A] As suggested, we have revised Figure 2 and added a new Example 2 to Section III-A to further illustrate dual patterns (pp. 4), highlighting the difference between “easy” and “hard” cases. Five dual patterns Q_1 to Q_5 are depicted in Figure 2. Note that the two star patterns in Q_1 , Q_3 and Q_4 have distinct topologies. Such dual patterns are able to capture features of entities with heterogeneous structures, e.g., two different paths in the two stars of Q_3 are used to get the manufacturers of accessories x_{a0} and y_{a0} , where one passes through the product collection node while the other does not. Dual patterns Q_2 and Q_5 are simple cases, bearing stars of identical structures.

[Availability] *The authors provide a Google Drive link as reference [53]. This does not work: because it is split on two lines, only the first part is a link, which goes to “Google Drive” ignoring the characters on the 2nd line; when copying it painfully by hand to put together the URI, an error 400 is encountered. The same or a similar link is provided independently as the reproducibility link to the paper. It allows to find the full report and some datasets. There does not seem to be available code nor instructions on how to reproduce the experiments. It would be better to point to a regular code deposit together with a Readme.*

[A] We have released the source codes on GitHub and changed the link to <https://github.com/hdqlkncso24jvf/LBoost>. A Readme file with detailed instructions on using the codes and an MIT License have also been provided for this repository.

[Additional remarks] *The authors write in III.B that each mapping serves as an explanation for the ML model prediction. This claim is a bit fuzzy; explainable ML is a field in itself, and there are proper ways of assessing if an “explanation” is a good one. I am not sure this is done in this paper. Maybe this claim could be either strengthened or dropped.*

[A] We have further clarified the following on explaining ML predictions with LBRs (Section III-B, pp. 5).

Given an ML model $\mathcal{M}$ for node classification or label cleaning, we can discover LBRs $Q[x_0, y_0](X \rightarrow \mathcal{M}(x_0, y_0) \geq \delta)$ from graph G . Here the dual pattern Q identifies the topologies and features that are relevant to the predictions, and

the predicates and conditions in X characterize $\mathcal{M}$ ’s overall behavior on graph G . Since 1-WL test (graded bisimilarity) is at least as expressive as node-classification GNNs [7], [8], [9], [10] (pp. 1), one can find pattern Q and precondition X (possibly with 1-WL test) that explain the prediction of $\mathcal{M}(x_0, y_0)$. Hence such LBRs can be treated as global explanations of $\mathcal{M}$ [11]. This is in contrast to most previous work on explaining ML models, e.g., [12], [13], [14], which consider local explanations at specific entities. Moreover, it has been shown that an explanation approach with similar logic rules achieves good fidelity and sparsity [11].

Many thanks for helping us improve the paper!

Response to the comments of Reviewer #2.

[O1&D1] *The effectiveness of LBoost depends on the quality of the LBRs mined by its Rule Discovery module; the impact that the module’s limitations might have on the overall approach would benefit from being investigated. The ablation study could be extended to show how LBoost’s accuracy is impacted by the quality of the mined LBRs.*

[A] Thanks! We have added a new ablation study to check the impact of rule quality (Exp-3 of Section VI, pp. 11).

Applying LBRs within different confidence intervals, the results in Figure 4(d) show that the accuracy generally increases when LBRs with higher confidence are used. Note that the improvement over citation networks Business and Computer and is not as substantial as over the e-commerce graphs Clothing, Sports and Office since their ambiguous class distributions result in less high-confidence rules.

[O2&D2] *The overhead incurred by the gradient-based method for computing probabilities for soft labels should be made explicit. It would be good to also report all the offline costs of LBoost when evaluating its performance for label imputation, also including the precomputation of graded bisimilarity relations from the training graph.*

[A] Thanks! We have reported in Exp-2 of Section VI (pp. 11) that over the real-life citation graphs and e-commerce graphs we used, it takes on average 731s and 77s, respectively, for deducing the probabilities of soft labels. Since almost all nodes in citation graphs represent publications that are to be classified, they need much longer soft-labeling time than e-commerce graphs that also include a number of entities other than products. The corresponding time for precomputing the graded bisimilarity relation is 1073s and 494s, respectively.

[O3&D3] *The generalizability of the approach to datasets of varying structural characteristics would require clarification. The representativeness of the chosen datasets should be expanded on. How does extending graded bisimilarity with feature vector similarity generalize to graphs of varying densities or label distributions, as well as to highly heterogeneous graphs?*

[A] As suggested, we have clarified the dataset characteristics

in the experimental setting (Section VI, pp. 9). We used six datasets from three different domains, including citation graphs Business and Computer, e-commerce graphs Clothing, Sports and Office, and a newly added social network Reddit. Among these, Reddit is substantially denser than the others, and Office is quite sparse. Regarding label distribution, social networks have relatively clear class boundaries, while the distributions of citation networks are more ambiguous, where classes often overlap. Hence their characteristics are diverse.

The experimental results throughout Section VI are consistent in general over the graphs with varying characteristics, including the newly tested dataset Reddit. These validate the generalization capability of our approach. Moreover, we have examined the effectiveness of the approach *w.r.t.* different label distribution in our ablation study (Section VI, pp. 11).

We have also clarified that our extension to the graded bisimilarity can be directly applied over graphs with various characteristics, including the heterogeneous ones (Section II-B, pp. 3). This is experimentally validated in Section VI (pp. 11), together with the improvement over the traditional notion of graded bisimilarity. Indeed, all the real-life datasets tested in our experiments are heterogeneous graphs with diverse entities, relationships and topological structures.

[O4&D4] *The paper would benefit from being more self-contained and further commenting on the practical applicability and potential limitations of the method. The TiE prediction rules, as well as the gist of GFD, GAR, and GCR would benefit from a bit more context, as currently these are mentioned but not explained.*

[A] As suggested, we have provided more explanations about previous graph rules in Section VII (pp. 12), including GFDs, NGFDs, GEDs, GARs, GCRs and TIEs. More specifically, a GFD is defined as $Q[\bar{x}](X \rightarrow Y)$, where Q is a generic graph pattern, and the dependency $X \rightarrow Y$ only allows predicates in the forms $x.A = y.B$ and $z.A = c$. NGFDs (resp. GEDs) extend GFDs by supporting arithmetic terms (resp. entity matching) in the dependencies, and GARs extend GEDs with ML and edge predicates. Both GCRs and TIEs adopt dual patterns, and combine single-valued logic predicates and ML models in the dependencies to clean graphs and improve recommendations, respectively. Beyond GCRs and TIEs, LBRs support set-valued predicates, graded bisimilarity, and ML models for label cleaning and node classification.

In Section VII (pp. 12), we have also discussed the limitations of this work. These include the need for parameter tuning in *e.g.*, rule discovery and soft labeling to get a good overall performance; and the requirements of initial ground truth Γ , *i.e.*, very inadequate Γ may reduce the accuracy.

[D5] *Can LBoost’s mining module further be optimized to eliminate the need for manual threshold tuning?*

[A] Thanks! We have clarified (Section IV, pp. 6) that one can tune the support and confidence thresholds via incremental rule-discovery algorithms in response to parameter updates [15]. As shown in [15], such incremental algorithms can

speed up batch discovery by up to two orders of magnitude. Hence, the users can start with parameter settings that are quick to discover an initial set of rules, and then gradually fine-tune the thresholds and incrementally mine rules that meet their need, based on the insights gained from the initial batch.

[D6] *Can LBoost be extended to support other GNN tasks, such as link prediction?*

[A] Yes, we have clarified that besides node-classification GNNs, LBoost supports other kinds of GNNs that can be characterized by 1-WL test [8], [9], [11], [10], *e.g.*, link prediction and recommendation (Section IV, pp. 6).

[D7] *How does the temperature parameter in the soft labeling strategy influence GNN training stability across the datasets?*

[A] As suggested, we have clarified this (Section V-B, pp. 8).

When the temperature parameter τ approaches 0, the probability distribution nearly resembles a deterministic hard label; this is more suitable for cases where each node belongs to a single class. On the other hand, the randomness of distribution increases when τ gets larger, which is most commonly used for datasets where nodes often span across multiple classes. In our experiments, we set $\tau = 1$ (Section VI).

[D8] *The data enrichment module depends on external data: what is the impact that misalignments between these and the considered graphs can have on the overall effectiveness of the method?*

[A] Thanks! In Section IV (pp. 6) and online Appendix A (pp. 16-17), we have clarified that the fine-tuned LLM effectively filters out irrelevant features from aligned nodes, mitigating the impact of misalignments between the graphs and external data. More specifically, we fine-tune the parameters of the LLM by treating the performance of $\mathcal{M}$ with different feature sets as learning signals. In other words, we use the given GNN model $\mathcal{M}$ as the evaluator to assess the efficacy of feature selection in a self-supervised way, and we retain only those features that reflect the essential semantics for classification with $\mathcal{M}$. In this way, irrelevant features of mismatched nodes from external data are disregarded. Hence misalignments have minimal impact on the overall accuracy.

[D9] *It would be interesting to comment on whether and how the approach could be adapted to support dynamic graphs.*

[A] Thanks! We have added a remark to Section IV (pp. 6). Employing the incrementalization techniques [16], [17], we can readily develop an efficient incremental label imputation algorithm in response to the updates to dynamic graphs, minimizing unnecessary recomputation.

Many thanks for your support and helpful suggestions!

Response to the comments of Reviewer #3.

[O1] *While I find the motivation convincing, I wonder why the application of graph data cleaning (rules) is not mentioned*

more prominently here. I think, explaining that the “logic rules” were originally developed for cleaning graph databases would improve the motivation for the database community.

[A] Thanks! In Section I (pp. 1), we have highlighted that over graphs, logic rules were originally developed for tasks such as cleaning graphs [18], [19], [20], [21], conducting association analyses [22], and enhancing recommendation systems [23].

[O2] Page 7, “The chase.”: I agree that is okay to refer to previous work for consequences that can also occur in rules defined in prior work (and it helps to reduce clutter). But, I suggest adding a short paragraph for 1-WL predicates, since they are introduced in this paper.

[A] Thanks! In Section V-A (pp. 7), we have clarified that no adjustments are needed when 1-WL predicates appear as consequences in LBRs. This is because 1-WL predicates are simply validated by the definition of graded bisimilarity.

[O3] Proposition 4: I assume it is known that C_m is a polynomial in $|G|$. I suggest to add a brief explanation/footnote or reference to strengthen the argument here (for those not familiar with this, like me).

[A] As suggested, we have added references to support the argument that C_m is a polynomial in $|G|$ for most common ML models [21], [24], [25] (pp. 8; online Appendix D, pp. 20).

As shown in [24], [25], the time complexity of ML prediction is a polynomial in the sizes of input data for convolution neural network, k-nearest neighbors, logistic regression, random forest and support vector machine.

[Availability] For reproducibility it would be very welcome to provide the source code permanently on a hosting service like GitHub or even better to archive the source code, for example via zenodo.org (the archive can be linked to GitHub). Using zenodo.org has the additional advantage of getting a DOI to cite the source code. A license for the code is not provided. This makes it difficult (and strictly speaking illegal in several countries) to reuse the code for (re)evaluation and comparisons in future work. I highly recommend to provide a license, see for instance <https://choosealicense.com/>.

[A] As suggested, we have released the source codes on GitHub (<https://github.com/hdqlkncso24jvf/LBoost>). An MIT license has also been provided for this repository.

[Additional remarks 1] page 2, the last sentence of “(3) System”: consider referring to the evaluation here to support

this claim

[A] Thanks! We have clarified that as shown in our experimental ablation study, a combination of the strategies adopted by LBoost consistently improves the accuracy of GNN-based node classification; any strategy alone does not suffice (pp. 2).

[Additional remarks 2] Definition of star patterns: I wonder whether it would simplify the definition if V_Q is defined as the set of variables (so there is no need for a bijection)

[A] As suggested, we have simplified the definition of star patterns and made the bijection implicit (Section III-A, pp. 3).

[Additional remarks 3] Figure 2 (and other examples): Consider using “more expressive” variable names, like b_0 , b_1 for nodes representing buyers

[A] Thanks! In Examples 1–4 (pp. 3–5, 7) and Figures 1–2 (pp. 3–4), we have renamed variables and nodes, e.g., x_{b_1} and x_{b_2} for variables representing buyers in dual patterns, and b_1 and b_2 for nodes that refer to buyers in the data graph.

[Additional remarks 4] Definition of matches: I suggest specifying vertex and edge set of G explicitly.

[A] As suggested, we have specified these sets explicitly in the definition of matches (Section III-A, pp. 4).

[Additional remarks 5] Definition of predicates: I think it should be $M(z, c)$ instead of $M(x, c)$?

[A] We have changed to $\mathcal{M}(z, c)$ throughout the paper.

[Additional remarks 6] Last sentence of “(2) Filling up unlabeled nodes”: I wonder if a ML predicate (a feature available for GARs) can be used to emulate a 1-WL predicate (so ϕ_2 could be expressed as a GAR)? If that makes sense, a brief (1 sentence) comment would be appreciated.

[A] As suggested, we have remarked in Section III-B (pp. 5) that if there exists an available ML model $\mathcal{M}$ having the same discriminative power as 1-WL test [10], then we can replace the $\Theta_{\simeq}$ operator with $\mathcal{M}$, and φ_2 can be expressed as a GAR.

[Additional remarks 7] page 6/Figure 3: renaming the heading “Label imputation (Section V)” to “Label Enhancement and Cleaning”, so it matches the figure (or vice versa)

[A] Figure 3 has been adjusted (pp. 6). Thanks!

Many thanks for your support and constructive suggestions!

Imputing Sparse and Noisy Labels for GNNs

Wenfei Fan^{1,2,3}, Kehan Pang¹, Chao Tian¹

¹Beihang University ²Shenzhen Institute of Computing Sciences ³University of Edinburgh
wenfei@inf.ed.ac.uk, {pangkehan, tianchao}@buaa.edu.cn

Abstract—This paper studies how to impute labels in training data of GNNs for node classification. We introduce Label Boosting Rules (LBRs), which extend graded bisimilarity and embed ML labeling models as predicates. With LBRs, we show how to (a) assign labels to unlabeled nodes via graded bisimilarity, which is at least as expressive as node-classification GNNs; (b) correct the labels of mislabeled nodes by both logic reasoning and ML prediction; (c) improve the accuracy of ML label cleaning with logic conditions; and (d) leverage the interaction of (a) and (b) to improve the overall labeling quality. We develop an algorithm to recursively rectify noisy labels and enhance sparse labels in a unified process; we show that the algorithm is Church-Rosser, tractable and parallelly scalable. We empirically verify that the method improves the accuracy of GNNs by 14.4% on average, up to 18.2%, and it scales with large graphs.

I. INTRODUCTION

Graph Neural Networks (GNNs) have found prevalent use in tasks such as recommender systems [26], [27], natural language processing [28], traffic forecasting [29], power systems [30], bioinformatics [31], [32], FinTech [33] and risk controlling [34] (see [35], [36] for surveys). In particular, node-classification GNNs divide the unlabeled nodes into predefined categories based on learned graph representations. They help predict *e.g.*, the topic of each article in citation networks [37] and the function role of each protein in protein-protein interaction networks [38]. After trained on properly labeled graphs, GNNs can accurately classify nodes *w.r.t.* their topological structures by aggregating information from their neighbors.

However, in real-life graphs, nodes are often mislabeled or unlabeled. For instance, in the social network Twitter (currently known as X), only 16% users register their city-level locations and 0.5% users post GPS tags in tweets, making it difficult to predict the geo-locations of users [39]. Noisy labels are prevalent even in popular benchmark datasets for graph learning [40], *e.g.*, at least 6.91% of data in the citation network PubMed [41] has been detected as mislabeled or ambiguous. In addition, more than 98.6% nodes are not associated with any label information within ogbn-papers100M [42]. Noisy and sparse labels substantially hamper the accuracy of GNNs. As observed in [43], GNNs often outfit noisy labels and propagate noise through the topology of graphs.

With this comes the need for imputing labels, to make GNN training robust against noisy and sparse labels. There are two issues here: (a) *label enhancement* to assign labels to unlabeled nodes, and (b) *label cleaning* to correct the labels of mislabeled nodes. Moreover, the two interact with each other. Indeed, after an unlabeled node is assigned a correct label, it can be used to fix the noisy labels of other nodes, which can

in turn enhance sparse labels. For example, when the subject area (label) of paper x_1 is filled up with value A , we can fix the incorrect subject of another paper x_2 with A if x_1 and x_2 have a number of references in common and are cited by a set of common papers in the same area. This in turn helps enhance an unlabeled paper x_3 with subject A if x_2 and x_3 have similar enough features derived from their titles and abstracts [42].

There have been a host of systems for “data cleaning for AI” [44], [45], [46], [47], [48], [49], [50], [51], [52], [53], [54], [55], to fix errors and noisy labels in training data, defend against adversarial attacks, and integrate multimodal data, mostly over tables, texts and images. However, the study of enhancing and cleaning labels for GNNs on graphs is still in its infancy [43], [56], [57], [58], [59]. Moreover, the prior approaches typically rely on data proximity [60], other trained machine learning (ML) models [58], or label distribution learning [61] to rectify noisy labels. They may suffer the same problems encountered when training GNNs.

Can we enhance and clean labels in a unified process, without training another ML model? Is it possible to combine data proximity and pre-trained ML models in the process, taking advantages of both, and further improving their accuracy? Can the integrated solution scale with large graphs in practice?

Contributions & Organization. This paper aims to answer the questions, by unifying data proximity, ML and logic rules.

(1) *Graded bisimilarity* (Section II). We extend the notion of graded bisimilarity [62], which is at least as expressive as node-classification GNNs [7], [8], [9], [10]. We extend the notion by supporting a similarity operator over the (possibly continuous) domain of feature vectors. The extension captures the structural proximity of nodes in real-life graphs, and helps label enhancement. Intuitively, if an unlabeled x_0 and a labeled y_0 are graded bisimilar, we can assign the label of y_0 to x_0 .

(2) *Rules* (Section III). We propose a class of Label Boosting Rules (LBRs). **Over graphs, logic rules were originally developed for tasks such as cleaning graphs [18], [19], [20], [21], conducting association analyses [22], and enhancing recommendation systems [23].** Departing from prior graph rules, LBRs may (a) take (extended) graded bisimilarity as a predicate for label enhancement, (b) embed pre-trained ML models for node classification or label cleaning as predicates, to reduce their false positives (FPs) and false negatives (FNs) with additional logic conditions; and (c) set-valued predicates to support soft labels, for modeling the distribution of labels and making GNNs more robust and accurate [63], [64], [65],

[66]. We strike a balance between the expressive power and complexity of LBRs by using special yet practical patterns and conditions. We show that their validation problem is in polynomial time (PTIME), for enhancing and cleaning labels.

(3) *System* (Section IV). We develop LBoost (Label Booster), a system for imputing labels for GNN classification. Given a graph G and a GNN $\mathcal{M}$, LBoost first (optionally) enriches G with external data and learns a set Σ of LBRs. It then imputes sparse and noisy labels by chasing G with Σ , guaranteeing the correctness. Using LBRs, it supports (a) label enhancement with graded bisimilarity, (b) label cleaning by unifying logic deduction and ML prediction, (c) reduction of FPs and FNs of ML labeling with logic conditions, and (d) soft labeling. It is the combination of the strategies that makes label imputation accurate. **As shown in our experimental ablation study, these strategies consistently improve the accuracy of GNN-based node classification; any strategy alone does not suffice.**

(4) *Label imputation* (Section V). Given the (possibly enriched) graph G and discovered LBRs Σ , LBoost extends the chase [67] on graphs with (a) a set semantics and (b) a gradient-based method, to support soft labels. It chases graph G with LBRs Σ , by accumulating and referencing ground truth Γ . It cleans noisy labels and enhances sparse labels in the same process, leverages their interaction to improve the overall quality, and recursively propagates the fixes to nodes in G . We show that the process is Church-Rosser, *i.e.*, it converges at the same set of fixes no matter what rules in Σ are used and in what order they are applied. Moreover, the process is in PTIME. Better yet, it is also parallelly scalable [68], *i.e.*, it guarantees to reduce runtime when given more processors.

(5) *Performance* (Section VI). Using real-life and synthetic graphs, we experimentally find the following. (a) LBoost consistently improves the accuracy of representative GNN classifiers: GCN [69], GraphSAGE [70] and GAT [71]. When 20% labels in the training graphs are noisy, the improvement is up to 17.5%, 17.7% and 18.2%, respectively. (b) In the same setting, it beats the state-of-the-art baselines by 5.6% in the percentage of accuracy improvement. (c) LBoost scales well with number of machines used. It gets $3.2\times$ faster when 16 machines are used instead of 4. **(d) All the strategies enforced by LBoost contribute to accuracy improvement without a dominant one.** (e) Data enrichment can effectively make LBoost more accurate with 3 features (attributes) added to each node.

We present related work in Section VII and future work in Section VIII. Data enrichment and proofs are deferred to [72].

II. GRADED BISIMILARITY

This section first reviews notations (Section II-A), and then presents an extension of graded bisimilarity (Section II-B).

A. Preliminaries

Assume three countably infinite alphabets Λ , Υ and Ω for tags, attributes and classes/labels, respectively.

Graphs. We consider directed graphs $G = (V, E, T, F_A)$, where (a) V is a finite set of nodes; (b) $E \subseteq V \times \Lambda \times V$ is a

finite set of edges, in which $e = (v, l, v')$ denotes an edge from node v to v' that is tagged with $l \in \Lambda$; (c) each node $v \in V$ has tag $T(v) \in \Lambda$; and (d) each node $v \in V$ carries a tuple $F_A(v) = (A_1, \dots, A_n)$ of *attributes* of a finite arity, where $A_i \in \Upsilon$ and $A_i \neq A_j$ if $i \neq j$, for properties. Different nodes may carry different attributes, not constrained by a schema.

A particular attribute is $v.\text{label}$ on each node v , denoting the *classification label* of v . In practice, v is often *unlabeled*, *i.e.*, $v.\text{label}$ is null, or *mislabeled*, *i.e.*, $v.\text{label}$ is put in a wrong class. *Label imputation* aims to instantiate $v.\text{label}$ with the correct label for unlabeled v , and rectify $v.\text{label}$ for mislabeled v .

Each attribute of v has a single value, except $v.\text{label}$ that may be a set of labels from Ω , *i.e.*, soft label (see below), each with a probability. Moreover, each node v is annotated with a feature vector $\mathcal{X}(v)$, which is derived from v 's attributes (excluding $v.\text{label}$) by an embedding method, *e.g.*, SimCSE [73].

A path ρ from a node v_0 in G is a list $\rho = (v_0, l_0, v_1, \dots, v_{n-1}, l_{n-1}, v_n)$ such that (v_{i-1}, l_{i-1}, v_i) is an edge in G ($i \in [1, n]$). In the sequel we consider *simple* paths on which each node appears at most once. A node v is called a *child* of u if there exists an edge (u, l, v) in the set E .

GNNs. Graph neural networks (GNNs) [43], [7] employ the graph topological structure and node features to learn a representation vector for every node, which can then be used for node classification and other tasks. Following the message-passing paradigm, GNNs iteratively update the representation of each node by aggregating representations of its neighbors across different layers. For the t -th iteration (layer of a GNN model), the updating process can be expressed as

$$\begin{aligned} \mathcal{G}^t(v) &= \text{AGGREGATE}^t(\{\mathcal{X}^{t-1}(v') \mid v' \in N(v)\}), \\ \mathcal{X}^t(v) &= \text{UPDATE}^t(\mathcal{X}^{t-1}(v), \mathcal{G}^t(v)), \end{aligned}$$

where AGGREGATE^t and UPDATE^t are aggregation and updating operations at the t -th iteration, respectively; $\mathcal{X}^t(v)$ is the feature vector of node v at the t -th layer, which is initialized as $\mathcal{X}^0(v) = \mathcal{X}(v)$; moreover, $N(v)$ denotes 1-hop neighborhood of v ; and $\mathcal{G}^t(v)$ is the aggregation result.

Stacking K layers for a predefined K , the GNN outputs the final representation $\mathcal{X}^K(v)$ for node v , and predicts its classification label as $g(\mathcal{X}^K(v))$. Here g is a function learned *w.r.t.* the (soft) labels in the training graph. Intuitively, after K iterations of aggregations, $\mathcal{X}^K(v)$ captures the structural information within K -hop neighborhood of node v , and separates distinct nodes if their neighborhoods are structurally different.

Training with soft labels. A *soft label* of a node v in a training graph of GNNs is a set $\{c_1, \dots, c_q\}$, where each c_i is a potential label of v from Ω ($i \in [1, q]$); c_i is associated with a probability z_v^i for how likely v belongs to class c_i , and $\sum_{i=1}^q z_v^i = 1$. Intuitively, soft labels catch the probability distribution in a set of classes, and accommodate multiple correct classes that semantically overlap, *e.g.*, Patek Philippe watches are classified under both wrist watches and luxury goods.

It has been shown that soft labels can smooth the training process [74], [75], *i.e.*, the parameters of GNNs evolve more smoothly, and distinguish ambiguous cases, making GNNs

more stable and robust [63], [64], [65], [66]. They help reduce overconfidence of the models and prevent overfitting, both of which are critical for improving model stability, especially when we have noisy labels or imbalanced data distribution.

B. Revising Graded Bisimilarity

We start with the conventional notion of graded bisimilarity [62]. Given a graph G and a pair (u, v) of nodes in V , u and v are *graded bisimilar* if there exists a binary relation R such that $(u, v) \in R$ and the following conditions are satisfied:

- *atom equality*: for all $(u', v') \in R$, $T(u') = T(v')$, i.e., the two nodes have the same tag;
- *graded forth*: for all $(u', v') \in R$ and all distinct children $u_1, \dots, u_k$ of node u' , there exist distinct children $v_1, \dots, v_k$ of node v' such that $(u_i, v_i) \in R$ for $i \in [1, k]$; and
- *graded back*: for all $(u', v') \in R$ and for all distinct children $v_1, \dots, v_j$ of v' , there exist distinct children $u_1, \dots, u_j$ of u' such that $(u_i, v_i) \in R$ for $i \in [1, j]$.

Intuitively, graded bisimilarity characterizes the 1-WL (1-dimensional Weisfeiler-Leman) test [76]. Starting with a graph in which all nodes get the same color, the 1-WL test works by color refinement as follows (cf. [7]): for all colors c in the current coloring and all nodes u and v of color c , u and v get different colors in the new coloring if there exists some color d such that u and v have different number of children of color d . This color refinement is iterated until no more changes can be made. That is, graded bisimilarity expresses 1-WL test in terms of topological matching, like graph simulation [77].

Extension. To better capture the structural proximity of nodes in real-life graphs and simulate the functionalities of GNNs, we revise the conventional graded bisimilarity by replacing the atom equality condition over a discrete tag domain with similarity checking over the domains of feature vectors.

Atom similarity. Consider two nodes u' and v' in graph G and an operator $\simeq$ based on a similarity metric over the domain of feature vectors, e.g., cosine similarity. We say that u' and v' are *atom similar* if $\mathcal{X}(u') \simeq \mathcal{X}(v')$. That is, the feature vectors of two nodes are “close” enough w.r.t. a predetermined threshold.

Then we change the atom equality condition of the conventional graded bisimilarity to *atom similarity* condition: for all $(u', v') \in R$, $\mathcal{X}(u') \simeq \mathcal{X}(v')$. The graded forth and graded back conditions remain the same as above.

Intuitively, we relax the tag equality checking to similarity validation. Since most GNNs in practice process feature vectors rather than tags, such an extension better captures GNNs. It allows us to handle more real-life graphs since the values of feature vectors, and even some attributes, are from large (continuous) domains. Indeed, we find that very few node pairs satisfy the atom equality condition. Instead, entities (e.g., products on e-commerce platforms) in the same class usually carry summary descriptions that yield numeric sentence vectors with high similarities [73]. Each element in these vectors is a learned real number (i.e., floating point value) that, when combined, encodes the semantic meaning of sentences. **Such an extension can be applied over graphs with**

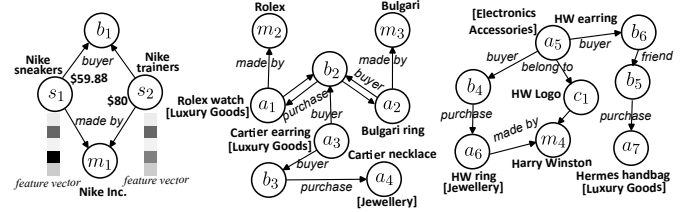


Fig. 1. Example graph G

various characteristics, including the heterogeneous ones.

Graded bisimilarity have the following properties.

- The extended graded bisimilarity can be computed by the 1-WL algorithm, in $O((|V| + |E|)\log|V|)$ time (cf. [7]).
- Node-classification GNNs are no more expressive than the 1-WL test when the initial feature vectors are consistent with initial coloring, i.e., two nodes u and v are given the same initial color if and only if $\mathcal{X}(u) = \mathcal{X}(v)$ [78], [10], [79], [80], and it is known that the 1-WL test is as expressive as graded bisimilarity [7], [8], [9], [10].

Example 1: A fraction G of a real-life e-commerce graph [81] is shown in Figure 1. Its nodes are tagged as follows: s_1 with sneakers, s_2 with trainers, b_1 to b_6 (resp. a_1 to a_7) with person (resp. accessory), m_1 to m_4 with athletic footwear company, watch company, fashion house and jewellery company, respectively, c_1 with product collection, and all available classification labels are given inside brackets.

A single-layer GNN classification model will put nodes s_1 and s_2 into the same product class since they both have features aggregated from **buyer b_1** and **manufacturer m_1** , and moreover, s_1 and s_2 bear very similar initial feature vectors (shown in Figure 1), which are derived from attributes of possibly continuous domains such as price and summary using certain embedding method. For the same reasons, s_1 and s_2 are graded bisimilar by the extended notion with atom similarity. In contrast, due to the distinct tags over s_1 and s_2 the two are not graded bisimilar by the conventional notion of [62]. $\square$

III. LABEL BOOSTING RULES

This section first defines LBRs (Section III-A) and then shows how LBRs impute labels (Section III-B). We also study the validation problem for LBRs (Section III-C).

A. Rules with Dual Star Patterns

We start with star-shaped pattern matching.

Pattern matching. LBRs are defined with dual star patterns.

Star patterns. A *star pattern* is $Q[x_0, \bar{x}] = (V_Q, E_Q, T_Q)$, where (1) V_Q is a finite set of pattern nodes, each uniquely identified by a variable in a distinct variable list $\bar{x}$; (2) E_Q is a finite set of pattern edges; (3) T_Q assigns a tag of Λ to each node in V_Q ; (4) x_0 is a designated variable in $\bar{x}$, referred to as the *center* of Q ; and (5) for each $z \in \bar{x}$, there exists a single path from x_0 to z , and z has at most one child, except x_0 . We also allow wildcard ‘_’ as a special tag in $Q[x_0, \bar{x}]$.

Intuitively, $Q[x_0, \bar{x}]$ has the shape of a star. The center x_0 denotes an entity of interest, and it links to a set of characteristic features (*leaf* nodes without children), each via a path.

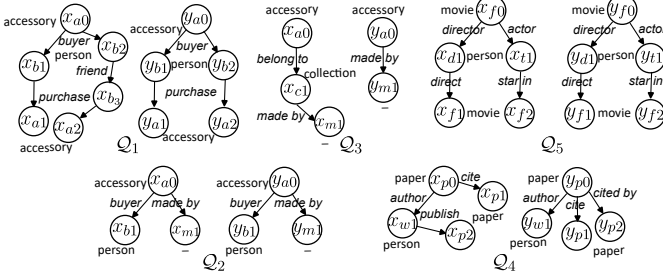


Fig. 2. Dual star patterns

Dual patterns. A *dual pattern* is defined as $\mathcal{Q}[x_0, y_0] = \langle Q_x[x_0, \bar{x}], Q_y[y_0, \bar{y}] \rangle$, where $Q_x[x_0, \bar{x}]$ and $Q_y[y_0, \bar{y}]$ are disjoint star patterns, i.e., Q_x and Q_y have no common nodes.

Intuitively, $\mathcal{Q}$ is a pair of patterns for two entities x_0 and y_0 , which specify features for pairwise comparison between x_0 and y_0 . They can also represent transformation of characteristic features by GNNs, where each feature is passed along a path from a leaf to x_0 or y_0 through (possibly) multiple GNN layers.

Example 2: Five dual patterns $\mathcal{Q}_1$ to $\mathcal{Q}_5$ are depicted in Figure 2. Note that the two stars in $\mathcal{Q}_1$, $\mathcal{Q}_3$ and $\mathcal{Q}_4$ have distinct topologies. Such dual patterns are able to capture features of entities with heterogeneous structures, e.g., two different paths in $\mathcal{Q}_3$ are used to get the manufacturers of accessories x_{a0} and y_{a0} , in which one passes through the product collection node while the other does not. Dual patterns $\mathcal{Q}_2$ and $\mathcal{Q}_5$ are simple cases, bearing stars of identical structures. $\square$

Matches. A match of a star pattern Q in a graph G is a homomorphic mapping h from the pattern nodes in Q to the node set V of G such that (a) for each node $u \in V_Q$, $T_Q(u) = T(h(u))$, and (b) for each pattern edge (u, l, u') in Q , $(h(u), l, h(u'))$ is in the edge set E of graph G . Here $T_Q(u) = T(h(u))$ if $T_Q(u)$ is ‘_’, i.e., wildcard matches an arbitrary tag.

Similarly, a match of a dual pattern $\mathcal{Q}[x_0, y_0] = \langle Q_x, Q_y \rangle$ in G is a homomorphic mapping h from pattern nodes in $V_{Q_x} \cup V_{Q_y}$ to the node set V of G . Here $h(x_0)$ and $h(y_0)$ are possibly disconnected in G . These allow us to compare entities that are not connected and may have different topological structures.

It has been shown that it is tractable to check the existence of matches of a star-shaped dual pattern [21]. In contrast, it is NP-complete for generic graph pattern matching (cf. [82]).

In the sequel, we refer to a star-shaped dual pattern simply as a *dual pattern* or a *pattern* when it is clear in the context.

Label Boosting Rules. We start with the predicates of LBRs.

Predicates. A predicate of $\mathcal{Q}[x_0, y_0] = \langle Q_x[x_0, \bar{x}], Q_y[y_0, \bar{y}] \rangle$ is

$$p ::= x.A \oplus y.B \mid z.A \oplus c \mid \Theta_{\simeq}(x, y) \mid \mathcal{M}(x, y) \odot \delta \mid$$

$\mathcal{M}(z, c) \odot \delta \mid x.\text{label} \otimes y.\text{label} \mid z.\text{label} = \{c\} \mid c \in z.\text{label}$, where $\oplus$ is one of $=, \neq, <, \leq, >, \geq$; $x \in \bar{x}$ and $y \in \bar{y}$ are variables in star patterns Q_x and Q_y , respectively, and $z \in \bar{x} \cup \bar{y}$; c is a constant; A and B are single-valued attributes in Υ , excluding label. Moreover, $\Theta_{\simeq}(x, y)$ returns true if and only if nodes x and y are graded bisimilar w.r.t. similarity operator $\simeq$, checking atom similarity on feature vectors; and $\mathcal{M}(x, y)$ (resp. $\mathcal{M}(z, c)$) is an ML model for label cleaning

Notations	Definitions
$\mathcal{Q}[x_0, y_0]$	dual pattern
$\mathcal{Q}[x_0, y_0](X \rightarrow p_0)$	a label boosting rule (LBR)
Σ	a set of LBRs
$v.\text{label}$	classification label of node v
$\Theta_{\simeq}(x, y)$	1-WL predicate
$\mathcal{M}(x, y) \odot \delta, \mathcal{M}(z, c) \odot \delta$	ML predicates
$x.\text{label} \otimes y.\text{label}, z.\text{label} = \{c\} \mid c \in z.\text{label}$	set predicates

TABLE I
NOTATIONS

or node classification that returns a numeric value in $[0, 1]$. In $\mathcal{M}(x, y) \odot \delta$, $\odot$ is $\geq$ or $<$, and δ is a predefined threshold in the range of $[0, 1]$; similarly for $\mathcal{M}(z, c) \odot \delta$. In $x.\text{label} \otimes y.\text{label}$, $\otimes$ is one of $=$ (set equality), $\subseteq$ (subset), and $\in$ (membership, when $x.\text{label}$ is a singleton and $y.\text{label}$ is a set).

Several ML models are already in place for label cleaning [43], [56], [57], [58], [59] and node classification [78], [79], [80], [83], [84], [85]. We consider such a model of the forms $\mathcal{M}(x, y)$ and $\mathcal{M}(z, c)$, where x, y and z are nodes, and c is a classification label. Here $\mathcal{M}(x, y)$ predicts that x and y are in the same class, and $\mathcal{M}(z, c)$ says that z is in class c .

We refer to (a) $\Theta_{\simeq}(x, y)$ as *1-WL predicate*, (b) $\mathcal{M}(x, y) \odot \delta$, $\mathcal{M}(z, c) \odot \delta$ as *ML predicates*, (c) $x.A \oplus y.B$, $\Theta_{\simeq}(x, y)$, $x.\text{label} \otimes y.\text{label}$ as *binary predicates*, (d) $z.A \oplus c$, $c \in z.\text{label}$, $z.\text{label} = \{c\}$, $\mathcal{M}(z, c) \odot \delta$ as *unary predicates*, and (e) $x.\text{label} \otimes y.\text{label}$, $z.\text{label} = \{c\}$, $c \in z.\text{label}$ as *set predicates*.

Note that x and y are variables for nodes in Q_x and Q_y , respectively, to pairwise compare features of centers x_0 and y_0 .

Rules. A *Label Boosting Rule* (LBR) φ has the following form:

$$\mathcal{Q}[x_0, y_0](X \rightarrow p_0),$$

where $\mathcal{Q}[x_0, y_0] = \langle Q_x[x_0, \bar{x}], Q_y[y_0, \bar{y}] \rangle$ is a dual pattern, X is a conjunction of predicates of $\mathcal{Q}$, and p_0 is a predicate of $\mathcal{Q}$. Moreover, in precondition X , (1) binary predicates are defined on two leaves or centers x_0 and y_0 in disjoint stars of $\mathcal{Q}$, respectively; and (2) each leaf may carry at most one binary predicate but multiple unary predicates. We refer to $\mathcal{Q}[x_0, y_0]$ and $X \rightarrow p_0$ as the *pattern* and *dependency* of LBR φ , and X and p_0 as *precondition* and *consequence* of φ , respectively.

Intuitively, (a) restrictions (1) and (2) above are to strike a balance between expressive power and efficiency (Theorem 1 and Corollary 2 below). (b) We separate $\mathcal{Q}$ and X in order to visualize the topological features of x_0 and y_0 by $\mathcal{Q}$, and speed up evaluation by leveraging the locality of pattern matching.

Example 3: Below are LBRs with the patterns of Example 2.

- (1) LBR $\varphi_1 = \mathcal{Q}_1[x_{a0}, y_{a0}](x_{a0}.\text{summary} = y_{a0}.\text{summary} \wedge x_{a1}.\text{label} \subseteq y_{a1}.\text{label} \wedge y_{a2}.\text{label} \subseteq x_{a2}.\text{label} \wedge x_{a2}.\text{score} = 5\text{-star} \wedge y_{a1}.\text{score} = 5\text{-star} \rightarrow x_{a0}.\text{label} = y_{a0}.\text{label})$. It classifies accessories x_{a0} and y_{a0} with the same label if their summaries match, and if the categories of (5-star) accessories x_{a1} and x_{a2} (resp. y_{a1} and y_{a2}) purchased by buyer x_{b1} and a friend x_{b3} of buyer (resp. y_{b1} and y_{b2}) are subsumed by each other.
- (2) LBR $\varphi_2 = \mathcal{Q}_2(\Theta_{\simeq}(x_{a0}, y_{a0}) \rightarrow x_{a0}.\text{label} = y_{a0}.\text{label})$. It categorizes accessories x_{a0} and y_{a0} into the same class, i.e., it assigns $y_{a0}.\text{label}$ to unlabeled x_{a0} if they are linked to certain buyers and manufacturers, and if they are graded bisim-

ilar *w.r.t.* the operator $\simeq$ that is defined with cosine similarity.

(3) LBR $\varphi_3 = \mathcal{Q}_3[x_{a0}, y_{a0}](\text{Luxury Goods} \in y_{a0}.\text{label} \wedge x_{a0}.\text{price} > 100K \wedge x_{m1}.\text{descr.} = y_{m1}.\text{descr.} \rightarrow \text{Luxury Goods} \in x_{a0}.\text{label})$. It enhances $x_{a0}.\text{label}$ with Luxury Goods, if its price is above 100K and if the description of its manufacturer x_{m1} matches that of y_{a0} , which is a known luxury good.

(4) LBR $\varphi_4 = \mathcal{Q}_4[x_{p0}, y_{p0}](\mathcal{M}(x_{p0}, y_{p0}) \geq \delta \wedge x_{p1}.\text{label} \subseteq y_{p1}.\text{label} \wedge y_{p2}.\text{label} \subseteq x_{p2}.\text{label} \rightarrow x_{p0}.\text{label} = y_{p0}.\text{label})$. Here $\mathcal{M}$ refers to the GNN model GCN [69]. This rule helps reduce the false positives (FPs) of GCN in classifying papers in terms of their themes. Besides important paper features, *e.g.*, titles and abstracts checked by $\mathcal{M}$, it additionally enforces that for two papers within the same class, the themes of the papers that they cite and cite them, and the themes of the authors' (other) publications should substantially overlap.

(5) LBR $\varphi_5 = \mathcal{Q}_5[x_{f0}, y_{f0}](x_{f1}.\text{label} = y_{f1}.\text{label} \wedge x_{f2}.\text{label} = y_{f2}.\text{label} \rightarrow \mathcal{M}(x_{f0}, y_{f0}) \geq \delta)$. Similar to φ_4 , here $\mathcal{M}$ is the GCN model. This rule interprets the prediction of GCN that classifies movies x_{f0} and y_{f0} to be of the same genre. This is because the two movies directed by the directors and starred by the actors of x_{f0} and y_{f0} also agree on the genres. It apprehends that GNNs can accumulate the information two-hops away from x_{f0} and y_{f0} to make predictions. $\square$

Semantics. Consider an LBR $\mathcal{Q}[x_0, y_0](X \rightarrow p_0)$. Denote by h a match of pattern $\mathcal{Q}$ in graph G , and by p a predicate of $\mathcal{Q}$.

A match h satisfies a predicate p , denoted by $h \models p$, if one of following conditions is satisfied. (a) When p is $x.A \oplus y.B$, the node $h(x)$ (resp. $h(y)$) carries attribute A (resp. B), and $h(x).A \oplus h(y).B$; similarly for $z.A \oplus c$. (b) When p is a 1-WL predicate $\Theta_{\simeq}(x, y)$, x and y are graded bisimilar *w.r.t.* operator $\simeq$. (c) When p is an ML predicate $\mathcal{M}(x, y) \odot \delta$, the ML model $\mathcal{M}$ predicts that x and y are in the same class with confidence score d , and $d \odot \delta$; similarly for $\mathcal{M}(z, c) \odot \delta$. (d) When p is $x.\text{label} \otimes y.\text{label}$, denote $h(x).\text{label}$ and $h(y).\text{label}$ by S_1 and S_2 , respectively, then $S_1 \otimes S_2$; similarly for $z.\text{label} = \{c\}$. (e) When p is $c \in z.\text{label}$, c is a member of the set $h(z).\text{label}$.

For a set X of predicates, we write $h \models X$ if match h satisfies *all* the predicates in X . We write $h \models X \rightarrow p_0$ if $h \models X$ implies $h \models p_0$. We say that a graph G satisfies an LBR $\varphi = \mathcal{Q}[x_0, y_0](X \rightarrow p_0)$, denoted by $G \models \varphi$, if for all matches h of $\mathcal{Q}[x_0, y_0]$ in G , $h \models X \rightarrow p_0$.

The notations of the paper are summarized in Table I.

B. Label Imputation

We now show general cases of label imputation by LBRs.

(1) *Rectifying mislabels.* LBRs of the form $\mathcal{Q}[x_0, y_0](X \rightarrow p_0)$ can rectify noisy labels, *e.g.*, φ_1 of Example 3, where p_0 is $x_0.\text{label} = y_0.\text{label}$ or $x_0.\text{label} = \{c\}$, and c indicates a class. That is, if precondition X holds, we can either let $x_0.\text{label}$ take correct $y_0.\text{label}$, or explicitly assign a right class c to $x_0.\text{label}$.

(2) *Filling up unlabeled nodes.* We can employ LBRs of the form $\mathcal{Q}[x_0, y_0](\Theta_{\simeq}(x_0, y_0) \rightarrow x_0.\text{label} \otimes y_0.\text{label})$ to propagate right labels to unlabeled nodes, *e.g.*, φ_2 . That is, if nodes x_0 and y_0 are graded bisimilar, when x_0 is unlabeled and y_0

is (correctly) labeled, then we can either assign the label of y_0 to $x_0.\text{label}$ with predicate $x_0.\text{label} = y_0.\text{label}$, or add $y_0.\text{label}$ to $x_0.\text{label}$ with $y_0.\text{label} \subseteq x_0.\text{label}$, making x_0 labeled.

If there exists an available ML model $\mathcal{M}$ having the same discriminative power as 1-WL test [10], we can replace the $\Theta_{\simeq}$ operator with $\mathcal{M}$, and φ_2 can be expressed as a GAR of [22].

One may also explicitly enhance $x_0.\text{label}$ with a specific label c by LBRs of the form $\mathcal{Q}[x_0, y_0](X \rightarrow p_0)$, where p_0 is either $x_0.\text{label} = \{c\}$ or $c \in x_0.\text{label}$.

(3) *Improving and leveraging ML models.* LBRs of the form $\mathcal{Q}[x_0, y_0](\mathcal{M}(x_0, y_0) \geq \delta \wedge X_1 \rightarrow x_0.\text{label} = y_0.\text{label})$ can typically reduce FPs of ML model $\mathcal{M}$ that is trained for node classification or label cleaning, *e.g.*, φ_4 . While $\mathcal{M}(x_0, y_0)$ predicts that x_0 and y_0 are in the same class (above threshold δ), we check additional conditions X_1 to filter FPs, *i.e.*, it overrides the classification of $\mathcal{M}$ if X_1 is not satisfied.

Similarly, $\mathcal{Q}[x_0, y_0](\mathcal{M}(x_0, y_0) < \delta \wedge X_1 \rightarrow x_0.\text{label} = y_0.\text{label})$ can reduce false negatives (FNs) of $\mathcal{M}$ for classification/labeling. While $\mathcal{M}(x_0, y_0)$ predicts that x_0 and y_0 are not in the same class (below threshold δ), if X_1 holds, then we override the prediction and put x_0 and y_0 in the same class.

Employing the improved $\mathcal{M}$, we fix unlabeled and noisy labels, without training another ML model starting from scratch.

(4) *Explaining ML predictions.* Given a GNN model $\mathcal{M}$ for node classification or label cleaning, we can discover LBRs $\mathcal{Q}[x_0, y_0](X \rightarrow \mathcal{M}(x_0, y_0) \geq \delta)$ from graph G , *e.g.*, φ_5 . Here the pattern $\mathcal{Q}$ identifies the topologies and features that are relevant to the predictions, and the predicates in X characterize $\mathcal{M}$'s overall behavior. Since 1-WL test is at least as expressive as node-classification GNNs [7], [8], [9], [10], one can find $\mathcal{Q}$ and X that explain the prediction of $\mathcal{M}$. Such LBRs provide global explanations of $\mathcal{M}$ [11], in contrast to, *e.g.*, [12], [13], [14], which consider local explanations at specific entities. Moreover, it has been shown that an explanation approach with similar logic rules achieves good fidelity and sparsity [11].

C. The Complexity of LBR Validation

To settle the complexity of label imputation with LBRs, we study the *validation problem* for LBRs, stated as follows.

- *Input:* A set Σ of LBRs and a graph G .
- *Question:* Does $G \models \varphi$ for all $\varphi \in \Sigma$?

The problem is NP-hard for GFDs, NGFDs, GEDs and GARs. Below we show that it becomes tractable for LBRs. While the problem is also tractable for GCRs [21], LBRs extend GCRs with graded bisimilarity and set predicates.

Theorem 1: The validation problem is tractable for LBRs. $\square$

Proof sketch: We will develop a PTIME algorithm for imputing labels with LBRs (Section V) by leveraging their star patterns and conditions (1) and (2) on preconditions X . It subsumes the validation problem for LBRs as a special case. $\square$

In contrast, relaxing any of the restrictions of LBRs makes the validation problem NP-hard. This is why we adopt dual star patterns and conditions (1) and (2) on X , to strike a bal-

ance between the expressive power and complexity of LBRs.

Corollary 2: *The validation problem of LBRs becomes coNP-hard when LBRs allow one of the following: (a) tree-shaped patterns; or (b) multiple binary predicates at each leaf.* $\square$

Proof sketch: The coNP-hardness holds since (1) GCRs are a special case of LBRs defined with predicates $x.A=y.B, z.A=c$ and $\mathcal{M}(x.\bar{A}, y.\bar{B})$ only; and (2) the validation problem for GCRs is coNP-hard if either (a) or (b) is allowed [21]. $\square$

IV. LBOOST: THE SYSTEM ARCHITECTURE

This section presents an overview of LBoost, our system for cleaning and enhancing labels for node-classification GNNs.

Architecture. As shown in Figure 3, to train a GNN model $\mathcal{M}$ for node classification, LBoost improves its training graphs G if their labels are sparse and/or noisy. It (optionally) enriches G with external data, and discovers a set Σ of LBRs from (samples of) G . Applying the LBRs of Σ to G , LBoost imputes labels in G . This generates cleaned G with correct soft labels. It provides GNN model $\mathcal{M}$ with the cleaned G as training data.

Key modules. We next present the main modules of LBoost.

Rule discovery. In this phase, LBoost mines a set of LBRs satisfying support and confidence thresholds. Here support and confidence are used to assess the quality of LBRs, a common practice for rule discovery. They indicate how often the LBRs can be applied in the graphs and how strong the association between precondition and consequence is, respectively.

LBoost employs the composite approach of [21] to mine patterns and dependencies of LBRs jointly in a single process, which already supports the discovery of dual patterns, and (unary and binary) predicates defined with single-valued attributes, constants and ML models. Set predicates on soft labels are handled along the same lines as for single-valued ones. The new addition is that LBoost precomputes the graded bisimilarity relation of nodes from the training graph in PTIME, upon which it readily treats 1-WL predicates as binary ones defined with two single-valued attributes like $x.A \oplus y.B$.

Label imputation (Section V). With this module, LBoost uniformly fills up sparse labels and cleans noisy labels in the same process, by chasing the graph G with the set Σ of discovered LBRs, accumulating and referencing a set Γ of ground truth.

LBoost applies the learned LBRs Σ to graph G to impute labels by adapting the chase [67]. This interleaves multiple tasks mentioned in Section III, *i.e.*, cleaning mislabels, enhancing unlabeled nodes, improving ML models, and unifying ML predictions and logic reasoning. Moreover, LBoost extends the classical chase with a set semantics and a gradient-based method to compute probabilities for soft labels.

Data enrichment. A challenge to label imputation is missing information. In practice, the graphs G are often not constrained by a schema and hence, important features for training a GNN $\mathcal{M}$ are sometimes missing from G . Moreover, GNN models may experience an accuracy drop of up to 50% due to 10% of missing node features [1]. To this end, LBoost also offers

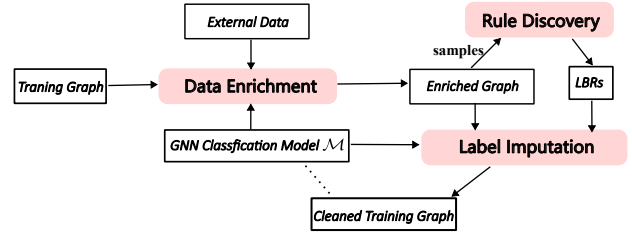


Fig. 3. The architecture of LBoost

an optional data enrichment module that extracts characteristic features from external data to help the training of $\mathcal{M}$.

We provide a data enrichment strategy (deferred to [72]). Guided by the performance of $\mathcal{M}$, we fine-tune a pre-trained large language model (LLM) to identify and extract characteristic features from $\mathcal{E}$ (*e.g.*, knowledge graph with overlapping information) that can improve the training of $\mathcal{M}$, and construct the enriched graph by including these features in G , without irrelevant features from $\mathcal{E}$. The enriched G is adopted as input in rule discovery and label imputation. Data enrichment often improves the semantics of data in G , and helps in both training model $\mathcal{M}$ and finding meaningful LBRs (see Section VI).

Remark. The discovery module can be extended to tune the support and confidence thresholds via incremental discovery upon parameter updates [15]. Users can start with parameter settings that are quick to discover an initial set of rules, and then gradually fine-tune the thresholds by incrementally mining rules that meet their need, based on the insights gained from the initial batch. In this way, LBRs can be incrementally learned *w.r.t.* the updated thresholds, with substantial speedup over batch mining. Moreover, the label imputation algorithm can be incrementalized to handle dynamic graphs [16], [17].

Besides node-classification GNNs, LBoost also supports other GNNs that can be characterized by 1-WL test [8], [9], [11], [10], *e.g.*, link prediction and recommendation.

V. LABEL IMPUTATION

This section provides a method for enhancing and cleaning labels with LBRs in the same process. To support soft labels, we extend the chase [67], [86] with (a) a set semantics (Section V-A), and (b) a gradient-based strategy to compute their probabilities (Section V-B). Based on the chase, we develop an imputation algorithm and prove its properties (Section V-C).

A. The Chase Revised

Given a training graph G and a set Σ of LBRs, we deduce “fixes” to impute the labels of nodes in G with Σ .

Fixes. Each fix has the form of $r_1 \circ r_2$, where $\circ$ is one of $=, \neq, <, \leq, >, \geq, \in, \subseteq$, and r_1 and r_2 are single-valued/attributes or constants. Intuitively, the fixes of $v_1.\text{label} = \{c\}$ and $c \in v_1.\text{label}$ (resp. $v_1.\text{label} = v_2.\text{label}$ and $v_2.\text{label} \subseteq v_1.\text{label}$) can expand $v_1.\text{label}$ with c (resp. $v_2.\text{label}$) to build soft labels. We refer to those fixes that are predetermined before the chase as *ground truth*, denoted as Γ . The ground truth can be collected (*e.g.*, sampled from the datasets, see Section VI) and validated by users, domain experts or crowd-sourcing, and is accumulated over time (see below).

We maintain a collection of sets $[v.\text{label}]_{\supseteq}$ to organize the node labels that are deduced as fixes. Each $[v.\text{label}]_{\supseteq}$ includes $v'.c$ such that $v'.\subseteq$ $v.c \in v.[v.\text{label}]_{\supseteq}$ collects (a) all nodes v' that should share classification labels with node v , and (b) all potential class values c for soft labels of v .

The chase. We impute labels in G by chasing the graph G with a set Σ of LBRs, and referencing ground truth Γ . To simplify the discussion, here all LBRs in Σ are assumed to have consequences in the form of $x_0.\text{label} = y_0.\text{label}$, $x_0.\text{label} = \{c\}$, $y_0.\text{label} \subseteq x_0.\text{label}$, or $c \in x_0.\text{label}$ for label imputation. The other kinds of LBRs can be applied in the chase along the same lines as in [86]. Then a *chase step* of G with Σ at a set $\bar{T}$ of fixes (i.e., a collection of sets $[v.\text{label}]_{\supseteq}$) is

$$\bar{T} \rightarrow_{(\varphi, h)} \bar{T}'.$$

Here φ is an LBR $\mathcal{Q}[x_0, y_0](X \rightarrow p_0)$ in Σ and h is a match of pattern $\mathcal{Q}[x_0, y_0]$ in the training graph G , such that

- (1) all predicates in the precondition X of LBR φ are *validated* by $\bar{T}$ at h ; that is, the instantiated fact, i.e., fix $h(p)$, is already included in $\bar{T}$ for each predicate p of X , where $h(p)$ replaces every variable x in p by $h(x)$; in particular, predicates on set-valued attributes, e.g., $c \in z.\text{label}$, are validated with the elements of sets $[x.\text{label}]_{\supseteq}$ in $\bar{T}$ based on *the set semantics*; ML predicates (resp. 1-WL predicates) are verified by definition *w.r.t.* the ML prediction results (resp. similarity among feature vectors); and
- (2) the consequence p_0 extends $\bar{T}$ to $\bar{T}'$ with a fix representing $h(p_0)$; that is, (a) if $h(p_0)$ is $v.\text{label} = \{c\}$ or $c \in v.\text{label}$, we add c to $[v.\text{label}]_{\supseteq}$; and (b) if $h(p_0)$ is $v.\text{label} = v'.v'.\subseteq$ $v.\text{label}$, we include $v'.[v.\text{label}]_{\supseteq}$ and copy all class values from $[v'. to $[v.\text{label}]_{\supseteq}$. We also make the relation formed by all $[v.\text{label}]_{\supseteq}$ in $\bar{T}'$ reflexive and transitive as described above.$

Note that no adjustments are needed when 1-WL predicates appear as consequences in LBRs, which are simply validated by the definition of graded bisimilarity, as stated earlier.

A *chasing sequence* ξ of G by (Σ, Γ) is $\bar{T}_0, \dots, \bar{T}_k$, where $\bar{T}_0$ is ground truth Γ , and for each $i \in [0, k-1]$, there exist an LBR $\varphi_i \in \Sigma$ and a match h_i of φ_i in G such that $\bar{T}_i \rightarrow_{(\varphi_i, h_i)} \bar{T}_{i+1}$ is a chase step of G with Σ at $\bar{T}_i$. We say that ξ *terminates* when no more LBRs in Σ can be applied to further extend $\bar{T}_k$ with new fixes. Such a sequence is referred to as a *terminal chasing sequence*. When a sequence terminates, we refer to $\bar{T}_k$ and the graph G updated with the fixes in $\bar{T}_k$ as the result of ξ .

Intuitively, the chase adopts the set semantics to monotonically expand $[v.\text{label}]_{\supseteq}$. Assume correct ground truth Γ and LBRs Σ . By induction on chase steps, one can verify that all the deduced fixes are logical consequences of Γ , Σ and predictions of ML models embedded in Σ , i.e., as long as Σ and Γ are correct and the ML models are accurate, so are the fixes. That is, all the deduced classification labels are possible and reasonable, and hence are kept in $[v.\text{label}]_{\supseteq}$. In particular, the wrong labels in $v.\text{label}$ do not have a chance to get into

$[v.\text{label}]_{\supseteq}$ when we start the chase with $\bar{T}_0 = \Gamma$, i.e., they are excluded by the chase and thus, are corrected. This is why we treat $v.\text{label} = v'.v'.\subseteq$ $v.\text{label}$ in the same way.

Example 4: Recall the graph G from Example 1, in which similar feature vectors are derived over m_2 and m_3 (resp. a_1 and a_2 ; not shown); consider a set Σ that consists of LBRs $\varphi_1 - \varphi_3$ given in Section III; and a block Γ of ground truth including the summary values on a_3 and a_5 that are identical, the 5-star values of score at a_2 and a_6 , a 150K value for the price of a_6 , and the classification labels at a_1, a_3, a_4, a_6 and a_7 .

(a) Observe that a_1 and a_2 have a common neighbor b_2 , they carry similar feature vectors, and the two are linked to m_2 and m_3 that have feature vectors of high similarity as well. Hence a_1 and a_2 are graded bisimilar in G by the extended notion of Section II; and we first fill in the missing label of a_2 with the value of Luxury Goods from a_1 by using LBR φ_2 .

(b) Applying LBR φ_1 , we next correct the label of a_5 , i.e., HW (Harry Winston) earring from Electronics Accessories to Luxury Goods, where the pattern node x_{a0} (resp. y_{a0}) of $\mathcal{Q}_1$ is mapped to a_5 (resp. a_3) in graph G .

(c) After that, due to the high price value of 150K, we enhance the label of a_6 with Luxury Goods by using LBR φ_3 .

Here enhancing unlabeled nodes and cleaning mislabels interact with each other. For instance, fixing the erroneous label in step (b) can be applied only if a_2 's label has been filled in step (a), which helps validate the predicate $x_{a1}.\text{label} \subseteq y_{a1}.\text{label}$ in the precondition of φ_1 . In contrast, other methods like NRGNN [43] do not rectify the incorrect label and may connect it to similar nodes for making pseudo labels. $\square$

The chase recursively propagates fixes by using enhanced or rectified labels to validate the preconditions of LBRs in subsequent steps for deducing more labels. It deduces soft labels for nodes in G when it terminates with the set $\bar{T}_k$ of fixes.

B. Deducing Probabilities

Different labels can be added to the same $[v.\text{label}]_{\supseteq}$ during the chase. To make them soft labels (Section II), we propose a gradient-based strategy *w.r.t.* the downstream GNN model $\mathcal{M}$ for computing the probabilities associated with these labels.

In practice, the training objective of a GNN model $\mathcal{M}$ typically aims to minimize a loss function, denoted by $L_{\mathcal{M}}$; one applies descent iteratively to find optimal values of the parameters of $\mathcal{M}$ that make the loss minimum. The gradient of $L_{\mathcal{M}}$ *w.r.t.* an input x , denoted as $\nabla_x L_{\mathcal{M}}$, represents how minor changes to x under the current parameter values of model $\mathcal{M}$ would affect the loss and the output of $\mathcal{M}$. By deducing soft label probabilities from such gradient values, it can be assured that small probabilities are associated to those labels that would inflict large and disruptive parameter updates of model $\mathcal{M}$. Then $\mathcal{M}$ is able to make stable and gradual adjustments by leaning towards smaller gradients only, which helps prevent overfitting and maintain the learned data representation during prediction refinement, thereby improving the generalizability.

In light of these, we adopt gradients to assess the confidence

Algorithm 1: LCleaner

Input: A (training) graph G , a set Σ of LBRs, ground truth Γ and GNN classification model $\mathcal{M}$.
Output: A set $\bar{T}_k$ of fixes and graph G updated with soft labels.

```

1  $\bar{T}_k \leftarrow \Gamma; \quad \bar{T}_d \leftarrow \Gamma; \quad \Sigma_d \leftarrow \emptyset;$ 
2 while  $\bar{T}_d$  is not empty do
3   foreach LBR  $\varphi \in \Sigma$  and fix  $s \in \bar{T}_d$  s.t.  $\varphi$  pertains to  $s$  do
4      $\Sigma_d \leftarrow \Sigma_d \cup \{\varphi\};$ 
5    $\bar{T}_d \leftarrow \text{LFix}(\Sigma_d, G, \bar{T}_k);$ 
6   update  $G$  in regards to  $\bar{T}_d$ ;  $\bar{T}_k \leftarrow \bar{T}_k \cup \bar{T}_d$ ;  $\Sigma_d \leftarrow \emptyset;$ 
7 Soft_Labeling( $\mathcal{M}, G$ );
8 return  $\bar{T}_k$  and  $G$ ;
```

of assigning different potential classification labels to nodes in the training data. More specifically, for the current values of parameters θ of $\mathcal{M}$ to classify a node v into class c_i , the parameter optimization via gradient descent can be expressed as

$$\Delta\theta_{(v,c_i)} = -\eta \nabla_{\theta} L_{\mathcal{M}}(v, c_i),$$

where $\nabla_{\theta} L_{\mathcal{M}}(v, c_i)$ represents the gradient of $L_{\mathcal{M}}$ w.r.t. parameters θ when changing v 's classification label to c_i , and η is the learning rate. The parameters of $\mathcal{M}$ are updated to minimize the loss in this case. Consequently, we use the L_2 -norm of $\Delta\theta_{(v,c_i)}$, denoted as $\|\Delta\theta_{(v,c_i)}\|_2$, to measure the confidence of classifying v into class c_i . Then larger L_2 -norm of $\Delta\theta_{(v,c_i)}$ indicates lower confidence of assigning potential classification label c_i to v , since such classification will cause a larger change to the parameters. Based on these L_2 -norms, the probability z_v^i of classifying node v into class c_i is computed as

$$z_v^i = \frac{e^{-\|\Delta\theta_{(v,c_i)}\|_2/\tau}}{\sum_{c \in [v.\text{label}]_{\geq}} e^{-\|\Delta\theta_{(v,c)}\|_2/\tau}}$$

when the chase terminates. Here τ is a temperature parameter that controls the smoothness and soft labels embed the probability. When the temperature parameter τ approaches 0, the probability distribution nearly resembles a deterministic hard label; this is more suitable for cases where each node belongs to a single class. On the other hand, the randomness of distribution increases when τ gets larger, which is most commonly used for datasets where nodes often span across multiple classes. In our experiments, we set $\tau = 1$ (Section VI).

C. Imputation Algorithm

We now develop an algorithm to implement the extended chase, denoted by LCleaner and shown in Algorithm 1. Its novelty includes (a) an iterative procedure to expand sets of values, rather than assigning a single value to each attribute [67]; (b) a soft labeling strategy to resolve the conflicts of conventional chase [86], by collecting rational labels; and (c) an incremental deduction method to reduce unnecessary chase steps. LCleaner is in PTIME and is Church-Rosser (see below).

LCleaner initializes $\bar{T}_k$ as the ground truth Γ , and the set $\bar{T}_d$ of newly deduced fixes also as Γ (line 1). Then it computes fixes and imputes labels iteratively until no new fixes can be found (lines 2–6). In each round, LCleaner finds from Σ the LBRs that pertain to the fixes deduced in last iteration and collects them in the set Σ_d (lines 3–4). An LBR φ pertains to a fix s if there exists a predicate p in φ 's precondition that has the same node tags, attributes and constant as s ; if p is an ML or 1-WL predicate, we check node tags of p only. In fact,

Graph	#Nodes	#Edges	Type
Business	0.9M	3.4M	Citation graph
Computer	3.8M	10.4M	Citation graph
Clothing	1.7M	2.5M	E-commerce graph
Sports	0.7M	1.1M	E-commerce graph
Office	0.5M	0.8M	E-commerce graph
Reddit	33K	0.2M	Social network

TABLE II
DATASETS

only such affected LBRs can contribute to deducing new fixes in the current round, where its chase steps need to validate the fixes s of last round, i.e., they are “activated” by s .

Using the LBRs in Σ_d selected for current round, LCleaner computes the set $\bar{T}_d$ of new fixes by calling procedure LFix (line 5, not shown). By our extended chase, it applies the LBRs in Σ_d one by one with the set semantics. To reduce redundant computation, here the matches of dual patterns are enumerated in an *incremental* manner, by firstly mapping to nodes that are involved in fixes derived from the last round. Then the graph G and the set $\bar{T}_k$ of fixes are adjusted with the new fixes after this step, where node labels in G are enhanced and fixed, i.e., updated with potential class values deduced in $\bar{T}_d$ (line 6). When the iterative process terminates, LCleaner computes the probabilities for soft labels for G using the gradient-based approach (line 7). Finally it returns the latest $\bar{T}_k$ and G with the soft labels (line 8).

The fixes $\bar{T}_k$ returned by algorithm LCleaner are accumulated in Γ to expand ground truth and facilitate subsequent label imputation with further runs of LCleaner.

Properties. LCleaner is Church-Rosser and tractable.

Church-Rosser. Under a certain condition on the ML models embedded in LBRs, the chase has the Church-Rosser property [67], [86] with the set semantics and soft labels, i.e., all chasing sequences of G by (Σ, Γ) converge at the same result, no matter what rules in Σ are used and in what order they apply.

The condition is that the ML models $\mathcal{M}$ in LBRs are robust to perturbations to node attributes [87]; i.e., their predictions remain unchanged in response to a small number of attribute value changes, i.e., $\mathcal{M}(x, y) \odot \delta$ (resp. $\mathcal{M}(z, c) \odot \delta$) holds on both graphs G and $G \diamond \Delta G$, where $G \diamond \Delta G$ denotes applying attribute changes ΔG to graph G . In practice, most node classification models and label cleaning models adopt the architecture of GNNs [43], [80]; as shown in [87], representative GNNs can be readily made robust against attribute changes,

Proposition 3: *Chasing with LBRs is Church-Rosser if their embedded ML models are robust to attribute changes.* $\square$

Complexity. By star patterns and restrictions on predicates, and PTIME for applying trained ML models [24], [25], we have:

Proposition 4: *Algorithm LCleaner is in PTIME.* $\square$

To scale with large graphs, we also parallelize LCleaner such that it guarantees to reduce runtime when more processors are available, i.e., it is parallelly scalable [68] (see [72]).

VI. EXPERIMENTAL STUDY

Using real-life and synthetic graphs, we experimentally evaluated (a) the effectiveness of LBoost in improving the accuracy of GNNs, (b) the efficacy and (parallel) scalability

Dataset	$\rho\%$	0%		5%							10%						
	Method	N/A	LBoost	N/A	KGClean	NRGNN	STRG	MDG	CLG	LBoost	N/A	KGClean	NRGNN	STRG	MDG	CLG	LBoost
Business	GCN	60.5	61.5	58.6	59.0	59.2	59.6	59.8	59.5	60.1	55.8	56.5	57.0	57.5	57.5	57.3	58.6
	GraphSAGE	60.5	61.2	58.3	58.5	58.9	59.3	59.3	59.5	59.8	55.5	56.1	56.5	57.3	57.1	56.8	58.3
	GAT	60.3	61.4	58.5	58.8	58.9	59.3	59.7	59.5	60.0	55.7	56.3	56.7	57.4	57.7	57.0	58.5
Computer	GCN	46.4	46.8	44.6	44.9	45.1	45.4	45.3	45.2	45.7	42.3	42.8	43.1	43.5	43.8	42.8	44.6
	GraphSAGE	46.0	46.7	44.5	44.8	44.9	45.3	45.3	45.2	45.6	42.2	42.5	43.1	43.1	43.8	43.1	44.5
	GAT	46.3	46.9	44.7	44.9	45.1	45.5	45.3	45.2	45.8	42.3	43.1	43.1	43.6	44.0	43.5	44.6
Clothing	GCN	70.8	71.8	67.8	68.0	68.0	69.4	69.4	68.4	69.5	63.5	64.0	64.0	65.4	66.1	65.0	67.6
	GraphSAGE	74.3	75.0	70.7	71.1	71.8	72.4	72.0	72.0	72.5	66.8	67.8	68.0	68.4	70.4	67.9	72.4
	GAT	74.0	75.2	70.5	70.9	71.0	71.9	72.2	71.7	72.4	66.4	67.5	67.6	67.4	68.7	67.8	70.7
Sports	GCN	71.1	72.0	67.6	68.1	67.7	68.4	69.1	68.0	70.0	63.5	64.0	63.8	64.1	65.2	64.3	68.8
	GraphSAGE	70.1	71.0	66.7	67.1	67.1	67.3	68.3	67.8	69.1	62.9	63.4	63.3	64.1	66.6	64.1	67.8
	GAT	69.1	70.3	66.0	66.7	66.1	67.0	67.6	67.4	68.2	62.2	62.8	62.2	63.4	65.2	63.7	66.8
Office	GCN	74.6	75.5	71.1	71.8	71.4	71.7	73.5	72.6	73.7	66.9	67.3	67.5	67.9	69.9	69.5	71.9
	GraphSAGE	79.0	79.9	75.2	75.7	75.5	75.8	76.7	75.2	78.0	70.9	71.8	72.0	71.7	73.8	73.5	76.2
	GAT	77.0	77.9	73.0	73.7	73.6	73.8	75.2	74.2	76.0	68.6	69.6	70.1	70.0	72.3	72.1	74.2
Reddit	GCN	61.5	62.6	60.5	61.1	60.9	61.5	61.6	61.6	61.8	59.6	59.9	60.2	60.5	60.6	60.5	60.9
	GraphSAGE	61.5	62.4	60.7	61.1	60.9	61.4	61.6	61.6	61.8	59.6	60.0	60.2	60.5	60.7	60.5	60.9
	GAT	61.7	62.6	60.8	61.2	61.1	61.5	61.8	61.7	62.0	59.9	60.2	60.5	60.4	60.7	60.7	61.2

Dataset	$\rho\%$	15%							20%						
	Method	N/A	KGClean	NRGNN	STRG	MDG	CLG	LBoost	N/A	KGClean	NRGNN	STRG	MDG	CLG	LBoost
Business	GCN	52.9	54.3	55.0	55.2	55.9	55.1	57.1	50.0	51.0	53.4	53.7	54.2	53.8	55.6
	GraphSAGE	52.6	53.9	54.7	54.8	55.2	54.7	56.9	49.7	51.7	53.2	53.2	54.0	53.3	55.3
	GAT	52.8	53.6	54.3	54.9	55.5	55.1	57.1	50.0	51.2	53.3	53.4	54.0	53.4	55.6
Computer	GCN	40.0	40.8	41.0	41.3	42.2	41.4	43.4	37.7	39.1	39.6	40.4	41.3	40.0	42.3
	GraphSAGE	40.0	40.6	40.9	41.1	41.9	41.3	43.3	37.7	38.9	40.0	40.4	41.0	40.3	42.2
	GAT	40.1	40.9	41.0	41.5	42.6	41.5	43.5	37.8	39.2	40.2	40.5	41.0	40.5	42.3
Clothing	GCN	60.3	62.5	62.7	64.1	64.4	63.8	66.8	56.0	57.4	58.8	60.0	63.1	62.7	64.3
	GraphSAGE	63.5	65.8	66.6	67.8	68.0	67.5	71.1	60.0	61.6	63.2	63.9	65.3	64.7	67.8
	GAT	63.2	65.6	65.9	66.7	67.2	66.7	70.0	59.0	61.2	63.0	63.7	65.8	64.0	67.5
Sports	GCN	60.3	61.6	62.8	63.8	64.8	63.8	66.1	56.7	60.4	62.1	63.4	64.2	63.5	65.8
	GraphSAGE	60.0	61.5	62.5	63.7	64.9	63.7	67.0	56.1	59.6	61.6	62.0	63.8	62.6	66.0
	GAT	59.0	61.1	61.8	63.2	64.0	63.2	66.0	55.5	59.5	61.1	61.9	62.7	61.9	65.0
Office	GCN	64.2	66.7	67.6	67.8	69.8	68.7	71.2	59.6	63.1	65.0	65.7	68.3	67.6	70.0
	GraphSAGE	67.0	68.7	69.1	70.5	73.6	72.1	75.4	63.4	65.2	65.8	68.6	72.6	71.0	74.2
	GAT	65.0	66.6	67.9	69.4	71.6	70.9	73.3	61.1	64.7	65.3	67.1	70.3	69.8	72.2
Reddit	GCN	57.4	58.0	58.0	58.5	58.5	58.4	59.1	56.5	56.9	56.9	57.5	57.8	57.5	58.5
	GraphSAGE	57.4	57.9	58.1	58.5	58.6	58.3	59.2	56.6	56.9	57.0	57.6	57.9	57.4	58.5
	GAT	57.5	58.0	58.1	58.7	58.8	58.5	59.2	56.6	57.1	57.0	57.5	58.0	57.6	58.7

TABLE III
THE ACCURACY OF GNNs ON NODE CLASSIFICATION WITH VARIOUS NOISES (MICRO-F1-SCORE (%))

of the label imputation method, and (c) the effectiveness of different components underlying LBoost. We defer a detailed evaluation of the data enrichment module to [72].

Experimental setting. We start with the experimental setting.

Experimental data. We used **six real-life graphs with diverse characteristics**, summarized in Table II. (1) Business and Computer are citation graphs from the Metadata-Aware Paper colLEction (MAPLE [88]), a benchmark for scientific literature tagging, *i.e.*, classifying scientific publications *w.r.t.* research topics. (2) Office, Sports and Clothing are e-commerce graphs extracted from Amazon [81], in which the product nodes are labeled with various categories. (3) **Reddit [89]** is a social network, for classifying whether a user (node) is within the top 50% popular, depending on the average score of the users’ published subreddits. Among these, Reddit is substantially denser than the others, and Office is very sparse. Regarding label distribution, social networks have relatively clear class boundaries, while the distributions of citation networks are more ambiguous, where classes often overlap.

Employing the Erdős-Reñyi model that is instrumental in random graph generation [90], we also generated larger synthetic graphs to test scalability, which have up to $0.6B$ nodes

and $0.4B$ edges. The tags, attributes and labels of synthetic graphs were randomly drawn from a domain of 100 symbols.

We randomly picked 5% of each graph as the training set. **Following the prior work [56], [43], [91], [92], we also corrupted the original labels which are assumed to be correct, using the poisoning method MGAttack [93].** It modified labels by an adversary before training, and was parameterized by a ratio $\rho\%$ of the noisy labels to all labels in the training set.

To collect the initial ground truth Γ for label imputation, we randomly sampled 25% (resp. 5%) of the available classification labels and attribute values from the training set of each real-life graph (resp. synthetic graph).

Data enrichment. By default, we equipped GNN training and LBoost with an enrichment strategy that adopts LLM fine-tuning (Figure 3; see details in [72]). **To facilitate such enrichment, we aligned entities in the real-life graphs tested with external data from OAG [2], [3], Amazon [4] and Wiki-data [5] as knowledge graphs, and chose Mistral-7B [6] as the base LLM for selecting characteristic features (attributes) that are helpful for simplifying classification boundary learning.** The module enhances the semantic information of real-life datasets with, *e.g.*, product descriptions, user comments for

e-commerce graphs, and user profiles for social networks.

ML Models. We considered three representative GNN models for the impact of label imputation on node classification: (1) GCN [69], by convolution-based aggregation; (2) GAT [71], by attention-based aggregation; and (3) GraphSAGE [70] that conducts aggregation based on neighborhood sampling. All GNNs were configured with 2 layers, a 256-dimensional embedding space, and a learning rate of 1×10^{-4} .

We adopted SimCSE [73] to derive feature vectors for nodes. We used GCN, GraphSAGE, GAT and Sentence-BERT [94] as ML predicates in LBRs, and reduce their FPs and FNs by adding logic conditions. Here Sentence-BERT classifies nodes solely based on the feature vectors.

Baselines. We implemented the rule discovery and label imputation modules of LBoost in C++, and the data enrichment module in Python. We compared LBoost with the following methods on improving the accuracy of GNNs. (1) KGClean [95] that uses graph embedding and active learning to detect and repair erroneous labels. (2) NRGNN [43] that reduces noisy labels by linking similar nodes and generating pseudo labels. (3) The self-training-based STRG [96] that trains GNNs with pseudo labels and clean local structures. (4) A mislabel detection method MDG based on early loss observation [92]. Unlike LBoost, it relies on intuitive observation and reasoning instead of semantically meaningful rules. (5) A confidence learning approach CLG that learns the joint label distribution [61] to distinguish between clean and noisy labels.

To conduct ablation study, we also considered the following variants of LBoost. (6) $LB_{w/oG}$, which does not incorporate graded bisimilarity. (7) $LB_{w/oS}$, which performs imputation with the conventional chase that handles one-hot-encoding hard labels only. (8) $LB_{w/oM}$, which only applies those LBRs that do not contain ML predicates and logic conditions simultaneously. (9) $LB_{w/oI}$, which conducts graded bisimilarity, soft labeling, and ML predictions separately.

Rules. Over synthetic graphs, real-life citation graphs and social networks, the predicates of discovered LBRs involve 6 attributes; for e-commerce graphs that exhibit a richer attribute set, we derived predicates from 10 attributes. We mined LBRs where each one has up to 5 predicates in its precondition. A total of 100 LBRs of high quality were reserved in each dataset.

Configuration. We used a cluster of up to 16 machines for conducting label imputation, each powered by 12 Intel Xeon 2.2GHz cores and 128GB DDR4 RAM. Unless stated otherwise, we used $n = 8$ machines in (parallel) label imputation, set $\rho\% = 0\%$, and employed the 100 discovered LBRs. In data enrichment, the fine-tuning and inference of the LLM were deployed over up to 4 consumer-level NVIDIA GeForce RTX 3090 GPUs, with one GPU used and 3 additional characteristic features (attributes) included by default. Each experiment was run 10 times; the average results are reported here.

Experimental results. We next report our findings.

Exp-1: Accuracy. We first evaluated the effectiveness of

Dataset	mask ratio (%)									
	0		10		20		30		40	
	N/A	LBoost	N/A	LBoost	N/A	LBoost	N/A	LBoost	N/A	LBoost
Business	60.5	61.5	59.9	61.2	57.3	59.8	55.6	58.1	53.9	56.9
Computer	46.3	46.8	44.2	45.1	43.2	44.3	42.7	44.0	42.4	43.8
Clothing	70.7	71.8	69.6	71.3	67.3	70.4	63.5	68.1	61.1	67.3
Sports	71.1	72	70.1	71.3	67.5	70.8	64.2	69.7	62.7	68.5
Office	74.6	75.5	72.5	74.9	70.7	74.6	69.2	73.9	66.8	72.4
Reddit	61.5	62.6	60.3	61.6	58.9	60.2	57.6	59.4	57.0	58.9

TABLE IV
THE ACCURACY OF GCN WITHOUT NOISES (MICRO-F1-SCORE (%))

different approaches in improving the accuracy of node-classification GNNs. Varying the ratio $\rho\%$ of corrupted labels from 5% to 20%, Table III reports the accuracy measured in terms of Micro-F1-Scores. We find the following.

(1) On average, LBoost improves the accuracy of GCN, GraphSAGE, and GAT by 7.4%, 7.7% and 7.6%, respectively, when the ratio $\rho\%$ varies from 5% to 20%. It exhibits higher improvements as $\rho\%$ increases, *e.g.*, the three GNNs become 12.5%, 12.5% and 12.7% more accurate on average, respectively, up to 17.5%, 17.7% and 18.2%, when $\rho\% = 20\%$.

(2) LBoost consistently outperforms all baselines on the three GNNs over six datasets. When $\rho\%$ is 20%, on average, it beats the baselines by 5.6% in terms of the percentage of accuracy improvement, *i.e.*, LBoost can further improve the accuracy of GNNs by 5.6% compared to the percentage achieved by the baselines. As opposed to other methods that overly rely on labeled data, LBoost not only leverages the (enriched) characteristic features that are helpful for GNN classification, but also applies (extended) graded bisimilarity to exploit the structural features for enhancing labels. A combination of the strategies contributes to a better efficacy.

(3) As expected, all approaches become less accurate with larger $\rho\%$. This said, LBoost shows the slowest decrease in accuracy in response to $\rho\%$. That is, LBoost is more accurate and robust to the level of noisy and sparse labels.

(4) LBoost improves the accuracy of these GNNs without noise injection (*i.e.*, $\rho\% = 0\%$) by 1.4%, 1.3% and 1.5%, respectively. Since all nodes in the training sets are already labeled (*i.e.*, no unlabeled nodes), the improvement is moderate.

Limited labels. We also examined the effectiveness of LBoost on training datasets with limited labels, a case closer to real life since nodes are often unlabeled in practice. Fixing $\rho\% = 0\%$, we masked labels with a ratio controlled from 0% to 40%. As reported in Table IV, LBoost improves the accuracy of GCN better as the amount of labeled data decreases. On average, it achieves a 4.0% improvement over real-life graphs when 20% labels are masked. These justify the need for applying LBRs when limited labeled data is provided without noises.

Exp-2: Label imputation. We next tested the scalability of the imputation method, and its sensibility to the number of rules.

Varying n . We varied the number n of machines used from 4 to 16. Over the six datasets, Figure 4(a) reports the elapsed time of the parallel label imputation module, *i.e.*, the parallelized algorithm of LCleaner, denoted as PLCleaner (see details

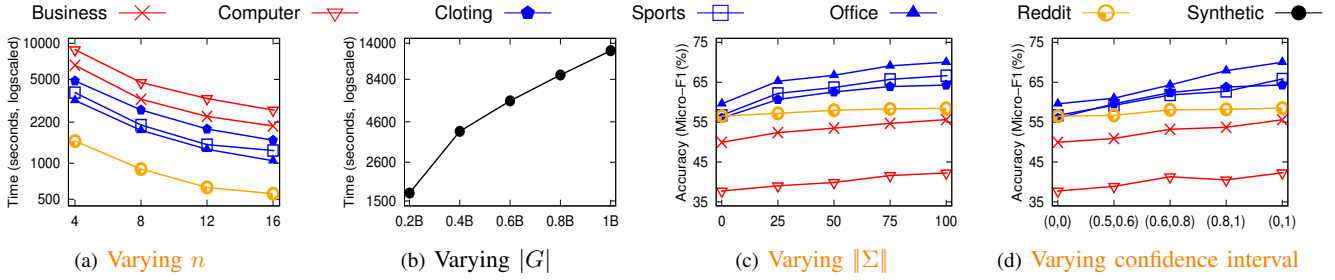


Fig. 4. Performance evaluation of label imputation

Dataset	N/A	LB _{w/oG}	LB _{w/oS}	LB _{w/oM}	LB _{w/oI}	LBoost
Business	55.8	57.6	57.9	56.8	56.2	58.6
Computer	42.3	43.7	44.0	43.5	43.2	44.6
Clothing	63.5	66.7	67.2	65.8	67.0	67.6
Sports	63.5	65.6	67.4	66.6	67.7	68.8
Office	66.6	70.7	70.1	69.3	70.5	71.9
Reddit	59.6	60.3	60.7	60.0	60.7	60.9

TABLE V

ABLATION STUDY ON IMPROVING GCN (MICRO-F1-SCORE (%))

in [72]). As shown there, PLCleaner scales well with n on all datasets. It achieves a speedup of $3.2\times$ on average when n is enlarged by $4\times$, validating the parallel scalability [68]. Hence in principle, we can effectively leverage parallelism to scale up the label imputation process for handling large graphs.

Varying $|G|$. Varying the size $|G| = |V| + |E|$ of synthetic graphs G from $0.2B$ to $1B$, we evaluated the scalability of PLCleaner using $n = 8$ machines. As shown in Figure 4(b), the larger $|G|$ is, the longer it takes, as expected. This said, PLCleaner works well on large-scale graphs, *e.g.*, it takes 4024s on graphs with 0.4 billion nodes and edges.

Varying $\|\Sigma\|$. Fixing $\rho\% = 20\%$, we varied the number $\|\Sigma\|$ of LBRs to check its impact on the accuracy of GCN. As reported in Figure 4(c), the accuracy gets higher as $\|\Sigma\|$ increases.

Remark. Over the real-life citation graphs and e-commerce graphs we used, it takes on average 731s and 77s, respectively, for deducing the probabilities of soft labels. Since almost all nodes in citation graphs represent publications that are to be classified, they need much longer soft-labeling time than e-commerce graphs that also include a number of entities other than products. The corresponding time for precomputing the graded bisimilarity relation is 1073s and 494s, respectively.

Exp-3: Ablation study. We next examined the effectiveness of different strategies adopted by LCleaner in improving the accuracy of GNN models. Fixing $\rho\% = 10\%$, we compared LCleaner against its variants over GCN in Table V.

The impact of graded bisimilarity. Comparing the improvement from LBoost and LB_{w/oG}, we find that the extended graded bisimilarity contributes an average of 1.8% to the accuracy improvement of GCN on node classification. Moreover, our extension to conventional graded bisimilarity with feature similarity helps enhance the accuracy by 2.1% (not shown). Since all the real-life datasets tested include diverse entities, relationships and topological structures, these validate the usefulness of the extended notion over heterogeneous graphs.

The impact of ML models. Table V also shows that LBoost beats LB_{w/oM} by 2.9% in terms of the percentage on

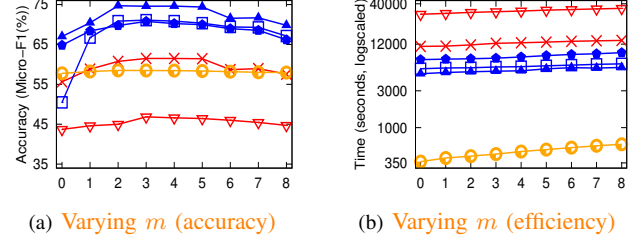


Fig. 5. Sensitivity to the number of added features (GCN)

improving GCN accuracy. This is due to the effective reduction of FPs and FNs predicted by embedded ML models, *e.g.*, GCN.

The impact of set predicates. Over the **six real-life graphs**, LBoost constantly performs better than LB_{w/oS} that only supports hard labels, where the set predicates contribute 1.4% to accuracy improvement. In fact, set-valued soft labels are beneficial as the conventional chase of LB_{w/oS} cannot correctly decide a proper classification label when conflicts arise. **Note that set predicates have minimal effect on Reddit because only two kinds of labels involve in the classification task for this graph.**

The impact of the interaction. From LBoost and LB_{w/oI}, we can see that combining (extended) graded bisimilarity, ML prediction and soft labels, on average, leads to a further 2.1% accuracy improvement for node classification with GCN. It verifies the importance of the uniform process.

These demonstrate the need for each of the strategies. They jointly make LBoost accurate without a dominant one.

The impact of rule quality. Fixing $\rho\%=20\%$, we used LBRs within various confidence intervals to check the impact. As reported in Figure 4(d) for GCN, the accuracy generally increases with higher-confidence LBRs. The improvement over citation networks Business and Computer is not as substantial as over the e-commerce graphs since their ambiguous class distributions yield less high-confidence rules.

The impact of data enrichment. To study the impact, we varied the number m of features (attributes) added to each node from external sources, where $\rho\%$ is set as 0%. Note that when $m = 0$, LBoost works without employing enrichment at all. As shown in Figure 5(a) for GCN, the accuracy first rises when m is varied from 0 to 3 or 4, and then decreases very slowly. The highest accuracy is observed at $m = 3$ or 4. This is because that too few features lead to insufficient information and over-fitting for the training, while too many features may introduce redundant data that is irrelevant to node classification.

For training time, Figure 5(b) shows that it exhibits a steady yet small increase as m gets larger. Hence LBoost is able

to improve the accuracy of GCN while warranting efficient training, by selecting only important features for classification.

The results on GraphSAGE and GAT are consistent.

Summary. We find the following. (1) Using LBRs, LBoost consistently improves the accuracy of GCN, GraphSAGE and GAT on node classification. When 20% labels in the real graphs are noisy, the improvement is 12.5%, 12.5% and 12.7% on average, respectively, up to 17.5%, 17.7% and 18.2%. It also achieves a 4.0% improvement when 20% labels are masked without noises. (2) It outperforms the baselines by 5.6% in the percentage of accuracy improvement of GNNs, when 20% of the labels are corrupted. (3) The label imputation module of LBoost is parallelly scalable: it is $3.2\times$ faster when 16 machines are used instead of 4. (4) All the strategies adopted contribute to accuracy improvement, and no one dominates. (5) The accuracy of LBoost can be effectively improved by the data enrichment module when 3 features are added.

VII. RELATED WORK

Data cleaning for AI. Previous work on the topic can be divided into cleaning for white-box models that targets specific downstream ML models, and black-box models that targets arbitrary models. (a) For white-box models, there has been work focusing on KNN classifiers [44], convex loss models [45], and differential models [46], [97], [47]. (b) For black-box models, BoostClean [48] proposes a statistical boosting framework to correct data errors; Picket [50] defends against data corruptions with a self-supervised deep learning model; ML-Clean [52], [53] integrates data cleaning, unfairness mitigation and data sanitization; TARS [98] applies oracle-based cleaning to handle poorly crowdsourced labels. Other efforts aim to enhance the parameter tuning [49] and understandability [54], [51] of the cleaning. (c) Related are also data collection for ML [99], [100], [55] and benchmark cleaning [101], [102].

Learning with noisy labels has been studied for images and texts. The work can be classified into (a) data proximity-based strategies [60], [103], which consider an instance as clean if its label is consistent with those of its K nearest neighbors; (b) ML-driven approaches [57], [104], [105]; and (c) logic-guided techniques, *e.g.*, Logic-LNCL [106] incorporates logic rules in learning from noisy labels, for text sentiment classification and named entity recognition; MisDetect [92] detects noisy labels that incur loss in the early stage of training for image and tabular data classification. Related is also confident learning [61] that directly estimates the joint label distribution of entire datasets to find label errors. There have also been ML-based attempts to make GNN training robust against sparse and noisy labels, including self-reinforcement supervision and consistency regularization [59], pseudo label generation [40], [43], [91], graph contrastive learning [107], label aggregation [56], and autoencoder-guided detection [58].

This work targets labels in graphs for GNNs. (1) It proposes to enhance labels by extending graded bisimilarity to characterize node-classification GNNs. (2) It imputes labels by unifying data proximity (extended graded bisimilarity), logic

reasoning and ML predictions in the same process, without training a new ML model. (3) It combines label enhancement and cleaning in the process, and explores their interaction to improve the overall labeling quality. (4) Its fixes to labeling are logical consequences of Σ , Γ and ML predictions, *i.e.*, the fixes are correct as long as the rules and ground truth are correct, and if the embedded ML models are accurate.

The limitations of this work include the need for parameter tuning in *e.g.*, rule discovery and soft labeling to get a good overall performance; and the requirements of initial ground truth Γ , *i.e.*, very inadequate Γ may reduce the accuracy.

Graph rules. Various rules have been studied for graphs, including functional dependencies for RDFs [108], [109], [110], [111], and (numeric) graph functional dependencies (GFDs, NGFDs) [18], [19], graph entity dependencies (GEDs) [20], graph association rules (GARs) [22], graph cleaning rules (GCRs) [21] and prediction rules (TIEs) [23] on property graphs. A GFD is defined as $Q[x](X \rightarrow Y)$, where Q is a generic graph pattern, and $X \rightarrow Y$ only allows equality predicates $x.A = y.B$ and $z.A = c$. NGFDs (resp. GEDs) extend GFDs by supporting arithmetic terms (resp. entity matching) in $X \rightarrow Y$, and GARs extend GEDs with ML and edge predicates. Both GCRs and TIEs adopt dual patterns, and they combine single-valued logic predicates and ML models to clean graphs and improve recommendations, respectively.

In contrast, (1) LBRs extend graded bisimilarity as a predicate to characterize node-classification GNNs, and adopt set-valued predicates to support soft labels. (2) Moreover, LBRs may embed existing ML models for label cleaning and node classification, and reduce their FPs and FNs by incorporating additional logic conditions over both single-valued and set-valued attributes. (3) Like GCRs and TIEs, LBRs adopt dual patterns to compare features of disconnected nodes in a schemaless graph, and star patterns instead of general patterns of GFDs, NGFDs, GEDs and GARs to strike a balance between the expressivity and complexity. As a consequence, LBR validation and label imputation are in PTIME, as opposed to NP-hard for GFDs, GEDs and GARs [18], [20], [22].

VIII. CONCLUSION

LBoost makes a first effort to impute labels by unifying data proximity, logic reasoning and ML predictions. (1) It proposes to enhance sparse labels by characterizing node-classification GNNs with extended graded bisimilarity, without training a new model. (2) It cleans mislabeled nodes by improving pre-trained ML models with logic conditions, to reduce FPs and FNs. (3) In a unified recursive process, it rectifies noisy labels and enhances sparse labels, leveraging their interaction to improve the overall quality; we show that the process is Church-Rosser, in PTIME and parallelly scalable. Our empirical study has verified that LBoost is promising in practice.

Topics for future work include extensions of LBoost to (a) support ML models beyond GNN, and (b) unify training, label imputing and data cleaning in the same (iterative) process, to improve the accuracy, fairness or robustness of ML models.

REFERENCES

- [1] E. Rossi, H. Kenlay, M. I. Gorinova, B. P. Chamberlain, X. Dong, and M. M. Bronstein, "On the unreasonable effectiveness of feature propagation in learning on graphs with missing node features," in *LoG*, 2022, p. 11.
- [2] F. Zhang, X. Liu, J. Tang, Y. Dong, P. Yao, J. Zhang, X. Gu, Y. Wang, E. Kharlamov, B. Shao, R. Li, and K. Wang, "OAG: Linking entities across large-scale heterogeneous knowledge graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 9225–9239, 2023.
- [3] F. Zhang, X. Liu, J. Tang, Y. Dong, P. Yao, J. Zhang, X. Gu, Y. Wang, B. Shao, R. Li, and K. Wang, "OAG: Toward linking large-scale heterogeneous entity graphs," in *SIGKDD*, 2019, p. 2585–2595.
- [4] "Amazon," 2024, <https://www.amazon.com/>.
- [5] "Wikidata," 2024, <https://www.wikidata.org/>.
- [6] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de Las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," *CoRR*, vol. abs/2310.06825, 2023.
- [7] M. Grohe, "word2vec, node2vec, graph2vec, x2vec: Towards a theory of vector embeddings of structured data," in *PODS*, 2020, pp. 1–16.
- [8] —, "The logic of graph neural networks," in *LICS*, 2021, pp. 1–17.
- [9] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe, "Weisfeiler and Leman go neural: Higher-order graph neural networks," in *AAAI*, 2019, pp. 4602–4609.
- [10] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *ICLR*, 2019.
- [11] W. Fan, L. Fan, D. Lin, and M. Xie, "Explaining GNN-based recommendations in logic," *PVLDB*, vol. 18, no. 3, pp. 715 – 728, 2024.
- [12] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GN-Explainer: Generating explanations for graph neural networks," in *NeurIPS*, 2019, pp. 9240–9251.
- [13] M. S. Schlichtkrull, N. De Cao, and I. Titov, "Interpreting graph neural networks for NLP with differentiable edge masking," in *ICLR*, 2020.
- [14] H. Yuan, H. Yu, J. Wang, K. Li, and S. Ji, "On explainability of graph neural networks via subgraph explorations," in *ICML*, 2021, pp. 12 241–12 252.
- [15] H. Chen, W. Fan, and J. Zheng, "Incremental rule discovery in response to parameter updates," *Proc. ACM Manag. Data*, 2025.
- [16] W. Fan and C. Tian, "Incremental graph computations: Doable and undoable," *ACM Trans. Database Syst.*, 2022.
- [17] W. Fan, M. Liu, S. Liu, and C. Tian, "Catching more associations by referencing knowledge graphs," *PVLDB*, 2024.
- [18] W. Fan, Y. Wu, and J. Xu, "Functional dependencies for graphs," in *SIGMOD*, 2016, pp. 1843–1857.
- [19] W. Fan, X. Liu, P. Lu, and C. Tian, "Catching numeric inconsistencies in graphs," *ACM Trans. Database Syst.*, vol. 45, no. 2, pp. 9:1–9:47, 2020.
- [20] W. Fan and P. Lu, "Dependencies for graphs," *ACM Trans. Database Syst.*, vol. 44, no. 2, pp. 5:1–5:40, 2019.
- [21] W. Fan, K. Pang, P. Lu, and C. Tian, "Making it tractable to detect and correct errors in graphs," *ACM Trans. Database Syst.*, vol. 49, no. 4, pp. 16:1–16:75, 2024.
- [22] W. Fan, R. Jin, M. Liu, P. Lu, C. Tian, and J. Zhou, "Capturing associations in graphs," *PVLDB*, vol. 13, no. 11, pp. 1863–1876, 2020.
- [23] L. Fan, W. Fan, P. Lu, C. Tian, and Q. Yin, "Enriching recommendation models with logic conditions," *Proc. ACM Manag. Data*, vol. 1, no. 3, pp. 210:1–210:28, 2023.
- [24] X. Zheng, J. Jia, S. Guo, J. Chen, L. Sun, Y. Xiong, and W. Xu, "Full parameter time complexity (FPTC): A method to evaluate the running time of machine learning classifiers for land use/land cover classification," *IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens.*, vol. 14, pp. 2222–2235, 2021.
- [25] B. Shah and H. Bhavsar, "Time complexity in deep learning models," *Procedia Computer Science*, vol. 215, pp. 202–210, 2022.
- [26] S. Wu, F. Sun, W. Zhang, X. Xie, and B. Cui, "Graph neural networks in recommender systems: A survey," *ACM Comput. Surv.*, vol. 55, no. 5, pp. 97:1–97:37, 2023.
- [27] C. Gao, Y. Zheng, N. Li, Y. Li, Y. Qin, J. Piao, Y. Quan, J. Chang, D. Jin, X. He, and Y. Li, "A survey of graph neural networks for recommender systems: Challenges, methods, and directions," *Trans. Recomm. Syst.*, vol. 1, no. 1, pp. 1–51, 2023.
- [28] L. Wu, Y. Chen, K. Shen, X. Guo, H. Gao, S. Li, J. Pei, and B. Long, "Graph neural networks for natural language processing: A survey," *Found. Trends Mach. Learn.*, vol. 16, no. 2, pp. 119–328, 2023.
- [29] W. Jiang and J. Luo, "Graph neural network for traffic forecasting: A survey," *Expert Syst. Appl.*, vol. 207, p. 117921, 2022.
- [30] W. Liao, B. Bak-Jensen, J. R. Pillai, Y. Wang, and Y. Wang, "A review of graph neural networks and their applications in power systems," *CoRR*, vol. abs/2101.10025, 2021.
- [31] X.-M. Zhang, L. Liang, L. Liu, and M.-J. Tang, "Graph neural networks and their current applications in bioinformatics," *Frontiers in genetics*, vol. 12, p. 690049, 2021.
- [32] H. Yi, Z. You, D. Huang, and C. K. Kwoh, "Graph representation learning in bioinformatics: Trends, methods and applications," *Briefings Bioinform.*, vol. 23, no. 1, 2022.
- [33] J. Wang, S. Zhang, Y. Xiao, and R. Song, "A review on graph neural network methods in financial applications," *CoRR*, vol. abs/2111.15367, 2021.
- [34] T. Liang, G. Zeng, Q. Zhong, J. Chi, J. Feng, X. Ao, and J. Tang, "Credit risk and limits forecasting in e-commerce consumer lending service via multi-view-aware mixture-of-experts nets," in *WSDM*, 2021, pp. 229–237.
- [35] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 32, no. 1, pp. 4–24, 2021.
- [36] W. Ju, Z. Fang, Y. Gu, Z. Liu, Q. Long, Z. Qiao, Y. Qin, J. Shen, F. Sun, Z. Xiao, J. Yang, J. Yuan, Y. Zhao, X. Luo, and M. Zhang, "A comprehensive survey on deep graph representation learning," *CoRR*, vol. abs/2304.05055, 2023.
- [37] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: large-scale information network embedding," in *WWW*, 2015, pp. 1067–1077.
- [38] J. Xu and Y. Li, "Discovering disease-genes by topological features in human protein-protein interaction network," *Bioinform.*, vol. 22, no. 22, pp. 2800–2805, 2006.
- [39] R. Li, S. Wang, and K. C. Chang, "Multiple location profiling for users and relationships from social network and content," *PVLDB*, vol. 5, no. 11, pp. 1603–1614, 2012.
- [40] Y. Li, M. Xiong, and B. Hooi, "GraphCleaner: Detecting mislabelled samples in popular graph learning benchmarks," in *ICML*, 2023, pp. 20 195–20 209.
- [41] Z. Yang, W. W. Cohen, and R. Salakhutdinov, "Revisiting semi-supervised learning with graph embeddings," in *ICML*, 2016.
- [42] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," in *NeurIPS*, 2020.
- [43] E. Dai, C. Aggarwal, and S. Wang, "NRGNN: Learning a label noise resistant graph neural network on sparsely and noisily labeled graphs," in *KDD*, 2021.
- [44] B. Karlaš, P. Li, R. Wu, N. M. Gürel, X. Chu, W. Wu, and C. Zhang, "Nearest neighbor classifiers over incomplete information: From certain answers to certain predictions," *arXiv preprint arXiv:2005.05117*, 2020.
- [45] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," *PVLDB*, vol. 9, no. 12, pp. 948–959, 2016.
- [46] L. Flokas, W. Wu, Y. Liu, J. Wang, N. Verma, and E. Wu, "Complaint-driven training data debugging at interactive speeds," in *SIGMOD*, 2022, pp. 369–383.
- [47] B. Hilprecht, C. Hammacher, E. Reis, M. Abdelaal, and C. Binnig, "DiffML: End-to-end differentiable ml pipelines," *arXiv preprint arXiv:2207.01269*, 2022.
- [48] S. Krishnan, M. J. Franklin, K. Goldberg, and E. Wu, "BoostClean: Automated error detection and repair for machine learning," *arXiv preprint arXiv:1711.01299*, 2017.
- [49] S. Krishnan and E. Wu, "AlphaClean: Automatic generation of data cleaning pipelines," *arXiv preprint arXiv:1904.11827*, 2019.
- [50] Z. Liu, Z. Zhou, and T. Rekatsinas, "Picket: Guarding against corrupted data in tabular data during learning and inference," *Vldb J.*, pp. 1–29, 2022.
- [51] B. Lockhart, J. Peng, W. Wu, J. Wang, and E. Wu, "Explaining inference queries with Bayesian optimization," *arXiv preprint arXiv:2102.05308*, 2021.
- [52] K. H. Tae, Y. Roh, Y. H. Oh, H. Kim, and S. E. Whang, "Data cleaning for accurate, fair, and robust models: A big data-AI integration approach," in *DEEM@SIGMOD*, 2019, pp. 1–4.
- [53] L. Flokas, W. Wu, J. Wang, N. Verma, and E. Wu, "How I stopped worrying about training data bugs and started complaining," in *DEEM@SIGMOD*, 2022, pp. 1–5.

- [54] H. Liu, C. Zhong, A. Alnusair, and S. R. Islam, "FAIXID: A framework for enhancing AI explainability of intrusion detection results using data cleaning techniques," *J. Netw. Syst. Manag.*, vol. 29, no. 4, 2021.
- [55] S. E. Whang, Y. Roh, H. Song, and J.-G. Lee, "Data collection and quality challenges in deep learning: A data-centric AI perspective," *VLDB J.*, pp. 1–23, 2023.
- [56] Y. Li, J. Yin, and L. Chen, "Unified robust training for graph neural networks against label noise," in *PAKDD*, 2021, pp. 528–540.
- [57] A. Iscen, G. Tolias, Y. Avrithis, O. Chum, and C. Schmid, "Graph convolutional networks for learning with few clean and many noisy labels," in *ECCV*, 2020, pp. 286–302.
- [58] B. Zhou, Y. Jiang, Y. Wang, J. Liang, J. Gao, S. Pan, and X. Zhang, "Robust graph representation learning for local corruption recovery," in *WWW*, 2023, pp. 438–448.
- [59] S. Qian, H. Ying, R. Hu, J. Zhou, J. Chen, D. Z. Chen, and J. Wu, "Robust training of graph neural networks via noise governance," in *WSDM*, 2023, pp. 607–615.
- [60] J. S. Sánchez, R. Barandela, A. I. Marqués, R. Alejo, and J. Badenas, "Analysis of new techniques to obtain quality training sets," *Pattern Recognit. Lett.*, vol. 24, no. 7, pp. 1015–1022, 2003.
- [61] C. G. Northcutt, L. Jiang, and I. L. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *J. Artif. Intell. Res.*, vol. 70, pp. 1373–1411, 2021.
- [62] M. Otto, "Graded modal logic and counting bisimulation," *CoRR*, vol. arXiv:1910.00039, 2020.
- [63] R. Müller, S. Kornblith, and G. E. Hinton, "When does label smoothing help?" in *NeurIPS*, 2019, pp. 4696–4705.
- [64] K. M. Collins, U. Bhatt, and A. Weller, "Eliciting and learning with soft labels from every annotator," in *HCOMP*, 2022, pp. 40–52.
- [65] Q. Nguyen, H. Valizadegan, and M. Hauskrecht, "Learning classification models with soft-label information," *J. Am. Medical Informatics Assoc.*, vol. 21, no. 3, pp. 501–508, 2014.
- [66] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [67] F. Sadri and J. D. Ullman, "The interaction between functional dependencies and template dependencies," in *SIGMOD*, 1980.
- [68] C. P. Kruskal, L. Rudolph, and M. Snir, "A complexity theory of efficient parallel algorithms," *TCS*, vol. 71, no. 1, pp. 95–132, 1990.
- [69] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [70] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *NIPS*, 2017, pp. 1024–1034.
- [71] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
- [72] "Code, datasets and full version," 2024, <https://github.com/hdqlkncso24jvf/LBoost>.
- [73] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *EMNLP*, 2021, pp. 6894–6910.
- [74] C.-B. Zhang, P.-T. Jiang, Q. Hou, Y. Wei, Q. Han, Z. Li, and M.-M. Cheng, "Delving deep into label smoothing," *IEEE Trans. Image Process.*, vol. 30, pp. 5984–5996, 2021.
- [75] X. Sun, B. Wei, X. Ren, and S. Ma, "Label embedding network: Learning label representation for soft training of deep networks," *arXiv preprint arXiv:1710.10393*, 2017.
- [76] B. Y. Weisfieler and A. A. Leman, "The reduction of a graph to canonical form and the algebra which appears therein," *NIT*, vol. 2, 1968.
- [77] R. Milner, *Communication and concurrency*. Prentice hall, 1989.
- [78] M. Hang, J. Neville, and B. Ribeiro, "A collective learning framework to boost GNN expressiveness for node classification," in *ICML*, vol. 139, 2021, pp. 4040–4050.
- [79] H. Jang, S. Park, S. Mo, and S. Ahn, "Diffusion probabilistic models for structured node classification," in *NeurIPS*, 2023.
- [80] J. Wu, J. He, and J. Xu, "Demo-net: Degree-specific graph neural networks for node and graph classification," in *KDD*, 2019, pp. 406–415.
- [81] "Amazon product graph," 2024, <https://cseweb.ucsd.edu/~jmcauley/datasets/>.
- [82] M. Garey and D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.
- [83] A. Chowdhury, S. Srinivasan, A. Mukherjee, S. Bhowmick, and K. Ghosh, "Improving node classification accuracy of GNN through input and output intervention," *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 1, pp. 17:1–17:31, 2024.
- [84] S. Maekawa, K. Noda, Y. Sasaki, and M. Onizuka, "Beyond real-world benchmark datasets: An empirical study of node classification with gnn," in *NeurIPS*, 2022.
- [85] Z. Wen, Y. Fang, and Z. Liu, "Meta-inductive node classification across graphs," in *SIGIR*, 2021, pp. 1219–1228.
- [86] W. Fan, P. Lu, C. Tian, and J. Zhou, "Deducing certain fixes to graphs," *PVLDB*, vol. 12, no. 7, pp. 752–765, 2019.
- [87] D. Zügner and S. Günnemann, "Certifiable robustness and robust training for graph convolutional networks," in *KDD*, 2019, pp. 246–256.
- [88] Y. Zhang, B. Jin, Q. Zhu, Y. Meng, and J. Han, "The effect of metadata on scientific literature tagging: A cross-field cross-model study," in *WWW*, 2023, pp. 1626–1637.
- [89] X. Huang, K. Han, Y. Yang, D. Bao, Q. Tao, Z. Chai, and Q. Zhu, "Can GNN be good adapter for LLMs?" in *Proceedings of the ACM on Web Conference 2024*, 2024, pp. 893–904.
- [90] S. Nobari, X. Lu, P. Karras, and S. Bressan, "Fast random graph generation," in *EDBT*, 2011, pp. 331–342.
- [91] J. Xia, H. Lin, Y. Xu, C. Tan, L. Wu, S. Li, and S. Z. Li, "GNN Cleaner: Label cleaner for graph structured data," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 2, pp. 640–651, 2024.
- [92] Y. Deng, C. Chai, L. Cao, N. Tang, J. Wang, J. Fan, Y. Yuan, and G. Wang, "MisDetect: Iterative mislabel detection using early loss," *PVLDB*, vol. 17, no. 6, pp. 1159–1172, 2024.
- [93] G. Liu, X. Huang, and X. Yi, "Adversarial label poisoning attack on graph neural networks via label propagation," in *ECCV*, 2022, pp. 227–243.
- [94] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese bert-networks," in *EMNLP-IJCNLP*, 2019.
- [95] C. Ge, Y. Gao, H. Weng, C. Zhang, X. Miao, and B. Zheng, "KGClean: An embedding powered knowledge graph cleaning framework," *CoRR*, vol. abs/2004.14478, 2020.
- [96] K. Li, Y. Liu, X. Ao, and Q. He, "Revisiting graph adversarial attack and defense from a data distribution perspective," in *ICLR*, 2023.
- [97] W. Wu, L. Flokas, E. Wu, and J. Wang, "Complaint-driven training data debugging for query 2.0," in *SIGMOD*. ACM, 2020, pp. 1317–1334.
- [98] M. Dolatshah, M. Teoh, J. Wang, and J. Pei, "Cleaning crowdsourced labels using oracles for statistical classification," *PVLDB*, vol. 12, no. 4, pp. 376–389, 2018.
- [99] Y. Roh, G. Heo, and S. E. Whang, "A survey on data collection for machine learning," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1328–1347, 2019.
- [100] G. Y. Lee, L. Alzamil, B. Doskenov, and A. Termehchy, "A survey on data cleaning methods for improved machine learning model performance," *arXiv preprint arXiv:2109.07127*, 2021.
- [101] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A benchmark for joint data cleaning and machine learning [experiments and analysis]," *arXiv preprint arXiv:1904.09483*, p. 75, 2019.
- [102] M. Abdelaal, C. Hammacher, and H. Schoening, "Rein: A comprehensive benchmark framework for data cleaning methods in ml pipelines," *arXiv preprint arXiv:2302.04702*, 2023.
- [103] R. Barandela and E. Gasca, "Decontamination of training samples for supervised pattern recognition methods," in *SSPR/SPR*, 2000, pp. 621–630.
- [104] X. You, L. Zhang, J. Wang, Z. Bao, Y. Wu, and S. Dong, "ENLD: Efficient noisy label detection for incremental datasets in data lake," in *ICDE*, 2023, pp. 1940–1952.
- [105] J. van Amersfoort, L. Smith, Y. W. Teh, and Y. Gal, "Uncertainty estimation using a single deep deterministic neural network," in *ICML*, 2020, pp. 9690–9700.
- [106] Z. Chen, H. Sun, H. He, and P. Chen, "Learning from noisy crowd labels with logics," in *ICDE*, 2023, pp. 41–52.
- [107] J. Yuan, X. Luo, Y. Qin, Y. Zhao, W. Ju, and M. Zhang, "Learning on graphs under label noise," in *ICASSP*, 2023, pp. 1–5.
- [108] A. Cortés-Calabuig and J. Paredaens, "Semantics of constraints in RDFS," in *AMW*, 2012.
- [109] W. Akhtar, A. Cortés-Calabuig, and J. Paredaens, "Constraints in RDF," in *SDKB*, 2010.
- [110] J. Hellings, M. Gyssens, J. Paredaens, and Y. Wu, "Implication and axiomatization of functional and constant constraints," *Ann. Math. Artif. Intell.*, vol. 76, no. 3–4, pp. 251–279, 2016.
- [111] W. Fan, Z. Fan, C. Tian, and X. L. Dong, "Keys for graphs," *PVLDB*, vol. 8, no. 12, pp. 1590–1601, 2015.

- [112] S. Bouraga, I. Jureta, S. Faulkner, and C. Herssens, “Knowledge-based recommendation systems: A survey,” *Int. J. Intell. Inf. Technol.*, vol. 10, no. 2, pp. 1–19, 2014.
- [113] B. Hu, C. Shi, W. X. Zhao, and P. S. Yu, “Leveraging meta-path based context for top-n recommendation with a neural co-attention model,” in *SIGKDD*, 2018.
- [114] H. Wang, F. Zhang, M. Zhang, J. Leskovec, M. Zhao, W. Li, and Z. Wang, “Knowledge-aware graph neural networks with label smoothness regularization for recommender systems,” in *SIGKDD*. ACM, 2019, pp. 968–977.
- [115] H. Wang, M. Zhao, X. Xie, W. Li, and M. Guo, “Knowledge graph convolutional networks for recommender systems,” in *WWW*, 2019.
- [116] Z. Wang, G. Lin, H. Tan, Q. Chen, and X. Liu, “CKAN: Collaborative knowledge-aware attentive network for recommender systems,” in *SIGIR*. ACM, 2020, pp. 219–228.
- [117] R. Xie, Z. Liu, H. Luan, and M. Sun, “Image-embodied knowledge representation learning,” *arXiv preprint arXiv:1609.07028*, 2016.
- [118] G. A. Gesese, R. Biswas, M. Alam, and H. Sack, “A survey on knowledge graph embeddings with literals: Which model links better literally?” *Semantic Web*, vol. 12, no. 4, pp. 617–647, 2021.
- [119] A. Kristiadi, M. A. Khan, D. Lukovnikov, J. Lehmann, and A. Fischer, “Incorporating literals into knowledge graph embeddings,” in *ISWC*, 2019.
- [120] W. Shen, J. Wang, and J. Han, “Entity linking with a knowledge base: Issues, techniques, and solutions,” *TKDE*, vol. 27, no. 2, pp. 443–460, 2015.
- [121] M. Chen, Y. Tian, K.-W. Chang, S. Skiena, and C. Zaniolo, “Co-training embeddings of knowledge graphs and entity descriptions for cross-lingual entity alignment,” *arXiv preprint arXiv:1806.06478*, 2018.
- [122] A. Vretinaris, C. Lei, V. Efthymiou, X. Qin, and F. Özcan, “Medical entity disambiguation using graph neural networks,” in *SIGMOD*, 2021, pp. 2310–2318.
- [123] Y. Liu, W. Shen, Y. Wang, J. Wang, Z. Yang, and X. Yuan, “Joint open knowledge base canonicalization and linking,” *CoRR*, vol. abs/2212.01207, 2022.
- [124] X. Wang, D. Wang, C. Xu, X. He, Y. Cao, and T.-S. Chua, “Explainable reasoning over knowledge graphs for recommendation,” in *AAAI*, 2019.
- [125] X. Wang, X. He, Y. Cao, M. Liu, and T.-S. Chua, “KGAT: Knowledge graph attention network for recommendation,” in *SIGKDD*, 2019.
- [126] X. Wang, T. Huang, D. Wang, Y. Yuan, Z. Liu, X. He, and T.-S. Chua, “Learning intents behind interactions with knowledge graph for recommendation,” in *WWW*, 2021.
- [127] M. A. Ahmad, Z. Borbora, J. Srivastava, and N. Contractor, “Link prediction across multiple social networks,” in *ICDM workshops*, 2010.
- [128] B. An, B. Chen, X. Han, and L. Sun, “Accurate text-enhanced knowledge graph representation learning,” in *NAACL-HLT*, 2018.
- [129] K. Toutanova, D. Chen, P. Pantel, H. Poon, P. Choudhury, and M. Gammon, “Representing text for joint embedding of text and knowledge bases,” in *EMNLP*, 2015.
- [130] S. Guo, Q. Wang, L. Wang, B. Wang, and L. Guo, “Knowledge graph embedding with iterative guidance from soft rules,” in *AAAI*, 2018.
- [131] P. Pezeshkpour, L. Chen, and S. Singh, “Embedding multimodal relational data for knowledge base completion,” *arXiv preprint arXiv:1809.01341*, 2018.
- [132] J. Kazama and K. Torisawa, “Exploiting Wikipedia as external knowledge for named entity recognition,” in *EMNLP-CoNLL*, 2007, pp. 698–707.
- [133] B. D. Trisedya, J. Qi, and R. Zhang, “Entity alignment between knowledge graphs using attribute embeddings,” in *AAAI*, vol. 33, no. 01, 2019, pp. 297–304.
- [134] X. Du, J. Yan, and H. Zha, “Joint link prediction and network alignment via cross-graph embedding,” in *IJCAI*, 2019.
- [135] X. Du, J. Yan, R. Zhang, and H. Zha, “Cross-network skip-gram embedding for joint network alignment and link prediction,” *TKDE*, 2020.
- [136] Y. Wang, Q. Yao, J. Kwok, and L. M. Ni, “Generalizing from a few examples: A survey on few-shot learning,” *ACM Comput. Surv.*, vol. 53, no. 3, pp. 63:1–63:34, 2021.
- [137] Z. Hu, Y. Dong, K. Wang, K.-W. Chang, and Y. Sun, “GPT-GNN: Generative pre-training of graph neural networks,” in *SIGKDD*. ACM, 2020, pp. 1857–1867.
- [138] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, “Graph contrastive learning with augmentations,” in *NeurIPS*, 2020.
- [139] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, “Strategies for pre-training graph neural networks,” *arXiv preprint arXiv:1905.12265*, 2019.
- [140] J. Qiu, Q. Chen, Y. Dong, J. Zhang, H. Yang, M. Ding, K. Wang, and J. Tang, “GCC: Graph contrastive coding for graph neural network pre-training,” in *SIGKDD*, 2020, pp. 1150–1160.
- [141] Y. Yin, Q. Wang, S. Huang, H. Xiong, and X. Zhang, “AutoGCL: Automated graph contrastive learning via learnable view generators,” *CoRR*, vol. abs/2109.10259, 2021.
- [142] J. Zhou, J. Shen, and Q. Xuan, “Data augmentation for graph classification,” in *CIKM*, 2020, pp. 2341–2344.
- [143] T. Zhao, Y. Liu, L. Neves, O. Woodford, M. Jiang, and N. Shah, “Data augmentation for graph neural networks,” in *AAAI*, vol. 35, no. 12, 2021, pp. 11015–11023.
- [144] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, “Deep graph contrastive representation learning,” *arXiv preprint arXiv:2006.04131*, 2020.
- [145] H. Gao, J. Li, W. Qiang, L. Si, F. Sun, and C. Zheng, “Bootstrapping informative graph augmentation via a meta learning approach,” *arXiv preprint arXiv:2201.03812*, 2022.
- [146] W. Fan, L. Geng, R. Jin, P. Lu, R. Tugey, and W. Yu, “Linking entities across relations and graphs,” in *ICDE*, 2022, pp. 634–647.
- [147] G. Papadakis, G. Mandilaras, L. Gagliardielli, G. Simonini, E. Thanos, G. Giannakopoulos, S. Bergamaschi, T. Palpanas, and M. Koubarakis, “Three-dimensional entity resolution with JedAI,” *Information Systems*, vol. 93, p. 101565, 2020.
- [148] R. Isele, A. Jentzsch, and C. Bizer, “Silk server-adding missing links while consuming linked data,” in *COLD*, 2010, pp. 85–96.
- [149] X. Fu, J. Zhang, Z. Meng, and I. King, “MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding,” in *WWW*, 2020, pp. 2331–2341.
- [150] G. Abuoda, S. Thirumuruganathan, and A. Aboulmaga, “Accelerating entity lookups in knowledge graphs through embeddings,” in *ICDE*, 2022, pp. 1111–1123.
- [151] K. Han, Y. Wang, C. Zhang, C. Li, and C. Xu, “Autoencoder inspired unsupervised feature selection,” in *ICASSP*, 2018, pp. 2941–2945.
- [152] M. A. Gordon, K. Duh, and N. Andrews, “Compressing Bert: Studying the effects of weight pruning on transfer learning,” *arXiv preprint arXiv:2002.08307*, 2020.
- [153] Y. Gao, N. Colombo, and W. Wang, “Adapting by pruning: A case study on BERT,” *arXiv preprint arXiv:2105.03343*, 2021.
- [154] C. Zhang, S. Wu, H. Zhang, T. Xu, Y. Gao, Y. Hu, and E. Chen, “NoteLLM: A retrievable large language model for note recommendation,” in *WWW*, 2024, pp. 170–179.
- [155] M. Karpinska and M. Iyyer, “Large language models effectively leverage document-level context for literary translation, but critical errors persist,” in *WMT*, 2023, pp. 419–451.
- [156] I. Dasgupta, A. K. Lampinen, S. C. Y. Chan, H. R. Sheahan, A. Creswell, D. Kumaran, J. L. McClelland, and F. Hill, “Language models show human-like content effects on reasoning tasks,” 2023.
- [157] X. Wang, W. Zhu, M. Saxon, M. Steyvers, and W. Y. Wang, “Large language models are latent variable models: Explaining and finding good demonstrations for in-context learning,” in *NeurIPS*, 2023.
- [158] M. Xu, T. Zhang, Z. Li, and D. Zhang, “InfoAT: Improving adversarial training using the information bottleneck principle,” *IEEE Trans. Neural Networks Learn. Syst.*, vol. 35, no. 1, pp. 1255–1264, 2024.
- [159] N. Tishby and N. Zaslavsky, “Deep learning and the information bottleneck principle,” in *ITW*, 2015, pp. 1–5.
- [160] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized LLMs,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [161] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [162] S. Dou, E. Zhou, Y. Liu, S. Gao, J. Zhao, W. Shen, Y. Zhou, Z. Xi, X. Wang, X. Fan, S. Pu, J. Zhu, R. Zheng, T. Gui, Q. Zhang, and X. Huang, “LoRAMoE: Revolutionizing mixture of experts for maintaining world knowledge in language model alignment,” *CoRR*, vol. abs/2312.09979, 2023.
- [163] N. Jain, P. Chiang, Y. Wen, J. Kirchenbauer, H. Chu, G. Somepalli, B. R. Bartoldson, B. Kailkhura, A. Schwarzschild, A. Saha, M. Goldblum, J. Geiping, and T. Goldstein, “NEFTune: Noisy embeddings improve instruction finetuning,” *CoRR*, vol. abs/2310.05914, 2023.
- [164] L. G. Valiant, “A bridging model for parallel computation,” *Commun. ACM*, vol. 33, no. 8, pp. 103–111, 1990.

Algorithm 2: Data enrichment process

Input: A graph G , a knowledge graph KG , a GNN classification model $\mathcal{M}$, a pre-trained lightweight LLM $\mathcal{L}$, and an HER function f .

Output: The enriched graph G .

```
1  $\mathcal{T}^c \leftarrow \emptyset$ ;  
2 foreach node  $u$  in  $G$  do /* First phase */  
3    $M[u] \leftarrow \{v \mid (u, v) \in f(G, KG)\}$ ;  
4   collect in  $\mathcal{T}^c(u)$  all features on  $M[u]$ ;  
5 Fine_Tune( $\mathcal{L}, G, \mathcal{T}^c, \mathcal{M}$ ); /* Second phase */  
6 foreach node  $u$  in  $G$  tagged by  $l_t$  do /* Third phase */  
7    $\mathcal{T}_{\mathcal{L}}(u) \leftarrow \text{Extract\_Feature}(\mathcal{L}, l_t, \mathcal{T}^c(u))$ ;  
8 update  $G$  with the features  $\mathcal{T}_{\mathcal{L}}$ ;  
9 return  $G$ ;
```

APPENDIX

A. Data Enrichment

This section develops a method that, given a graph G , a knowledge graph KG with overlapping information, and a GNN classification model $\mathcal{M}$ to train, extracts relevant features from KG and enriches G , to improve the accuracy of $\mathcal{M}$.

Data enrichment aims to enhance a dataset by extracting additional information from external sources. It has been adopted in recommender systems [112], [113], [114], [115], [116], link prediction [117], [118], [119], [120], and entity resolution (ER) [121], [122], [123]. Recommenders, *e.g.*, KPRN [124], Kgat [125] and KGIN [126], learn embeddings of user-item relationships from external knowledge bases. Link prediction and ER leverage features extracted from social graphs [127], texts [128], [129], pre-computed rules [130] and knowledge graphs [131], [132], [133]. Cross-graph embedding was used to integrate link prediction and network alignment [134], [135]. Related is also few-shot learning [136]. In particular, training GNNs requires abundant task-specific labeled data, which is often obtained at high cost [137], especially for biological and chemical domains [138], [139]. To address this issue, data enrichment has been adopted in graph learning such as pre-training and self-supervision [138], [139], [140].

One might want to employ existing graph enrichment strategies such as [141], [142], [143], [144], [145]. However, such methods often adopt feature engineering but do not aim to improve the performance of the downstream models. Moreover, they consider only existing features within the graphs, although real-life data keeps evolving and may reside in external sources. The problems become more evident when it comes to graphs with noisy and sparse labels.

In light of these, we propose to enrich graph G with relevant and important features from an external source KG by using lightweight large language models (LLMs), guided by the downstream GNN model $\mathcal{M}$. LBoost enriches graph with features for both labeling and training the given GNN $\mathcal{M}$.

The integration of KGs and LLMs, unlike encoder-only models, facilitates the direct extraction of structured information from rich unstructured or semi-structured text attributes in KGs. Our method below supports flexible incorporation

of KGs tailored to various contexts and downstream tasks, from general-purpose KGs like Wikidata [5] to domain-specific ones, *e.g.*, Amazon [4] and OAG [2], [3].

Workflow. As shown in Algorithm 2, our enrichment process extracts features from KG via heterogeneous entity resolution (HER) [146], fine-tunes a pre-trained lightweight LLM as the feature selector, and employs the LLM to identify features (attributes) that are relevant and helpful to the given GNN model $\mathcal{M}$. We add such features to G , yielding enriched G .

Phase 1: HER (lines 2–4). We employ a function f for HER to retrieve candidate features for each node u in training graph G . Function f takes graph G and knowledge graph KG as input, and identifies node pairs (u, v) such that u in G and v in KG refer to the same real-world entity. We refer to v as an *aligned* node of u by f . Intuitively, if u and v pertain to the same entity, then they have the same tag and we can enrich u with relevant features of its aligned node v . Thus, all the aligned nodes of u by f are added to the set $M[u]$ (line 3). We also use another set $\mathcal{T}^c(u)$ to maintain the candidate features *w.r.t.* tag $T(u)$, including raw features (attributes) from KG (*i.e.*, aligned nodes with tag $T(u)$) (line 4).

Several HER functions are already in place, *e.g.*, JedAI [147], parametric simulation [146], and ML models Silk [148], MAGNN [149] and EMBLOOKUP [150]. In this paper, we take parametric simulation [146] as HER, which takes $O(|G||KG|)$ time to compute all matched pairs across G and KG . Our method works with the other HER functions.

Phase 2: LLM fine-tuning (line 5). We fine-tune the parameters of a lightweight LLM $\mathcal{L}$ with the collected features in a self-supervised manner, guided by the input GNN model $\mathcal{M}$. This fine-tuning is to ensure that the LLM $\mathcal{L}$, treated as a feature selector for the raw candidate feature set $\mathcal{T}^c$, can pick out characteristic features that help the training of model $\mathcal{M}$.

Phase 3: Feature selection (lines 6–7). For each node u in G , among the raw features of u extracted from KG , we select those that are predicted helpful by the fine-tuned $\mathcal{L}$ during LLM inference, denoted as $\mathcal{T}_{\mathcal{L}}(u)$. We keep only the characteristic ones $\mathcal{T}_{\mathcal{L}}(u)$ in the initial feature sets, which can be used to generate feature vectors (Section II). We enrich the input graph G with such selected features.

Note that the fine-tuned LLM effectively filters out irrelevant features from aligned nodes, substantially mitigating the effects of misalignments between the graphs and external data. In other words, unnecessary features of mismatched nodes from external data are disregarded. Hence misalignments have minimal impact on the overall accuracy.

We now elaborate LLM fine-tuning and use (phases 2–3).

Feature selection. Prior approaches often adopt, *e.g.*, sparse autoencoder [151] or BERT [152], [153], based on an encoder-only architecture. However, such methods either do not comprehend well the semantics embedded in textual data when working on features generated from texts of (knowledge) graphs, or fail to recognize the relationships hidden in the

unlabeled data [154]. In contrast, decoder-only LLMs have proven effective in understanding semantic associations and extracting crucial information from textual data [155], [156], [157]. The self-attention mechanism underlying LLMs can weight elements and discern how relevant features in the input sequence are to the current task, and help select critical features adaptively. This is an application of the information bottleneck principle [158], [159], which guides LLMs to retain features that are closely related to the output.

In light of these, **we propose to eliminate unnecessary features for GNNs based on LLMs. To better adapt a pre-trained LLM $\mathcal{L}$ for a given GNN model $\mathcal{M}$, we fine-tune the parameters of $\mathcal{L}$ by treating the performance of $\mathcal{M}$ with different feature sets as learning signals** (line 5 of Algorithm 2). We also make the training (a) cost-effective, and (b) robust against noisy and sparse labeled data.

Efficient fine-tuning. The paradigm for applying an LLM $\mathcal{L}$ for feature selection in graphs can be modeled as

$$\mathcal{L}(\theta, D(\mathcal{T}^c), \mathcal{T}^c(u)) \Rightarrow \mathcal{T}_{\mathcal{L}}(u) = [\mathbf{t}_u^{s_1} : \mathbf{t}_u^{s_2} : \dots : \mathbf{t}_u^{s_m}].$$

That is, given the input raw candidate feature set $\mathcal{T}^c(u)$ for node u which may include unstructured or semi-structured text attributes, we select m characteristic features with a configurable parameter m . The output $\mathcal{T}_{\mathcal{L}}(u)$ is represented as a concatenation of selected features, where $\mathbf{t}_u^{s_i}$ denotes the s_i -th crucial feature (attribute) derived from the raw information of u . Moreover, θ denotes the parameters of the LLM $\mathcal{L}$, and $D(\mathcal{T}^c)$ indicates the examples that guide the LLM (see the code repository in [72] for detailed examples used).

Then the embedding model $\mathcal{M}_e$, e.g., SimCSE [73], can be applied to derive the feature vector embedding $\mathcal{X}_{\mathcal{L}}(u)$ for each node u as $\mathcal{X}_{\mathcal{L}}(u) = [\mathcal{M}_e(\mathbf{t}_u^{s_1}) : \mathcal{M}_e(\mathbf{t}_u^{s_2}) : \dots : \mathcal{M}_e(\mathbf{t}_u^{s_m})]$. As explained in Section II, such feature vectors of a fixed dimension that is irrelevant to m will be used as the initial input to the GNN classification model $\mathcal{M}$.

We adopt a lightweight LLM as $\mathcal{L}$, e.g., Mistral-7B [6]. We apply QLoRA [160] to fine-tune $\mathcal{L}$ in a parameter-efficient way, such that the training can be executed with a *single consumer GPU*, e.g., NVIDIA GeForce RTX 3090 GPU (see details in Section VI). QLoRA, with the LoRA [161] technique, adjusts the parameters θ of LLM $\mathcal{L}$ by introducing quantized low-rank matrices instead of directly modifying the original pre-trained parameters, thereby preventing catastrophic forgetting while enabling efficient fine-tuning [162]. That is, for each linear transformation matrix W in $\mathcal{L}$, QLoRA approximates changes in the original transformation through the incorporation of two low-rank matrices A and B , expressed as $\Delta W = AB^T$, where the dimensions of A and B are significantly smaller than W . Moreover, QLoRA implements 4-bit and double quantization techniques, which reduce the memory footprint and computational load. These allow us to perform effective tuning with fewer parameter updates, while keeping the architecture of $\mathcal{L}$ unchanged.

Improving robustness. We sample a set V_t of nodes u from graph G to construct the training data for LLM fine-tuning. However, the nodes may carry noisy and sparse labels, even

with no explicit annotations indicating feature importance. To rectify these and improve the robustness of the LLM, we introduce noise vectors to the feature sets. This helps reduce the effects of noisy data and prevent from overfitting *w.r.t.* specific characteristics of the data, along the same lines as NEFTune [163]. Moreover, **we use the given GNN model $\mathcal{M}$ as the evaluator to assess the efficacy of feature selection in a self-supervised way.** More specifically, we first let

$$\mathcal{T}_{\mathcal{L}}(u) = \mathcal{L}(\text{Tokenizer}_{\mathcal{L}}(\mathcal{T}^c(u)) + \frac{\alpha}{\sqrt{L \times d}} \times \epsilon),$$

where u is a node in G ; ϵ is a noise vector sampled from the uniform distribution $U(-1, 1)$; L and d denote the feature sequence length and embedding dimension, respectively; these two parameters are used to control noise scale, such that the impact of noise on embedding vectors of different lengths and dimensions can be assessed; α is a tunable parameter, which allows the LLM $\mathcal{L}$ to dynamically adjust the intensity of noise based on the training data, thus achieving more precise adaptation to the downstream classification task. Then the parameters θ of the LLM $\mathcal{L}$ are optimized by

$$\theta = \arg \min - \sum_{u \in V_t} \log[\hat{y}_u^{\mathbb{1}(u, \mathcal{M})} (1 - \hat{y}_u)^{1 - \mathbb{1}(u, \mathcal{M})}],$$

where $\hat{y}_u = \text{Prob}_{\theta}(\mathcal{M}(\mathcal{X}_{\mathcal{L}}(u)) = y(u) | \mathcal{T}_{\mathcal{L}}(u))$, and $\mathbb{1}(u, \mathcal{M})$ is an indicator function defined as

$$\mathbb{1}(u, \mathcal{M}) = \begin{cases} 1, & \mathcal{M}(\mathcal{X}_{\mathcal{L}}(u)) = y(u); \\ 0, & \text{otherwise.} \end{cases}$$

Indeed, the outputs of the input GNN model $\mathcal{M}$ are encoded by function $\mathbb{1}(u, \mathcal{M})$. Our objective is to maximize the probability that the LLM-selected features yield the correct prediction the same as the classification achieved by $\mathcal{M}$.

Observe that we compare the outputs of the GNN model $\mathcal{M}$ before and after feature selection via self-supervised learning. This helps the LLM $\mathcal{L}$ retain those features that reflect the essential semantics for classification with $\mathcal{M}$. In this way, we also enhance $\mathcal{L}$ in its ability to understand complex inter-feature correlations. Consequently, even with a limited number of labeled training instances, the fine-tuned $\mathcal{L}$ is able to effectively identify characteristic features for classification with $\mathcal{M}$.

Semantic extraction. Based on the predictions of the fine-tuned LLM $\mathcal{L}$, for each node u , we can determine the characteristic features from its raw feature set $\mathcal{T}^c(u)$ *w.r.t.* the specific node tag (lines 6–7 of Algorithm 2). More specifically, fixing parameter m , the fine-tuned $\mathcal{L}$ outputs the top- m features from $\mathcal{T}^c(u)$ that are most important for GNN $\mathcal{M}$ during inference, which are hence treated as characteristic ones. However, in practice, it is common that values are missing for certain node attributes, even in knowledge graph KG, e.g., the information about less well-known people in KG is likely to be sparse. When this happens, besides selecting characteristic features, the LLM $\mathcal{L}$ also generates semantically meaningful contents for those features that are identified to be important but are missing from any non-blank node. For instance, it can produce the keywords for a paper based on its available abstract. Such semantic extraction in data

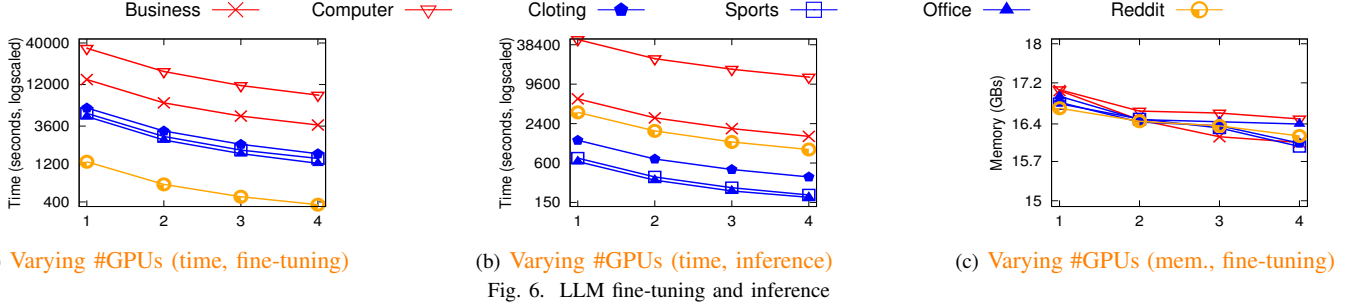


Fig. 6. LLM fine-tuning and inference

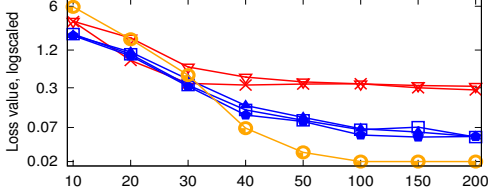


Fig. 7. Training loss of LLM

enrichment also helps the downstream classification.

Example 5: Given graph G of Example 1, knowledge graph KG and GNN model $\mathcal{M}$, we first match nodes across G and KG by HER. For instance, w_2 and w_3 are aligned with entities that represent Rolex SA and Bulgari S.p.A. in the KG, respectively. Then the fine-tuned LLM $\mathcal{L}$ identifies two characteristic features that are most critical for classifying w_2 and w_3 w.r.t. their tags and model $\mathcal{M}$ if m is set as 2, namely, style and price level. The same values of these features for w_2 and w_3 will be extracted from KG and added to G , i.e., luxury design and very high prices, making an enriched graph. $\square$

Experiments. We evaluated the time, memory consumption and the training convergence of LLM fine-tuning and inference, the main component in the data enrichment module.

Varying #GPUs. We varied the number of GPUs (consumer-level NVIDIA GeForce RTX 3090) adopted, from 1 to 4 to check its impact on the efficiency and memory usage. As shown in Figures 6(a)–6(b) over the six real-life graphs, the fine-tuning and inference of the lightweight LLM $\mathcal{L}$ almost achieve a linear speedup with the increase of GPUs used. They are on average $3.7\times$ and $3.7\times$ faster, respectively, when 4 GPUs are used instead of 1. The fine-tuning is efficient in practice, taking less than 2 hours with 2 GPUs in most of the cases.

Figure 6(c) depicts the memory usage per GPU in fine-tuning. We can see that less memory is used in fine-tuning when more GPUs are available. This is due to the reduced amount of data distributed to each GPU. In contrast, we find that the memory consumption in inference stays at a stable level of around 21.6GB on each GPU (not shown), since our inference framework employs the technique of virtual memory, i.e., physical memory consumption remains stable even when there are changes to the size of data.

Training loss. We tested the convergence of LLM fine-tuning. As shown in Figure 7, the training loss decreases very quickly in the first 40 to 50 training steps and converges at a low value within only 100 training steps. Hence, it is practical for LLM $\mathcal{L}$ to adapt to the task of feature selection guided by GNNs.

The tests above verify that data enrichment with LLM fine-tuning and inference is a practical and scalable solution.

B. Proof of Corollary 2

We prove the coNP-hardness results based on following.

- (1) Graph cleaning rules (GCRs) [21] are a special case of LBRs defined with binary predicates $x.A = y.B$, $\mathcal{M}(x.\bar{A}, y.\bar{B})$ and unary predicates $z.A = c$ only. They adopt the same type of dual patterns of a star form as LBRs.
- (2) The validation problem of GCRs has already been shown coNP-hard when they support (a) tree patterns instead of star patterns, or (b) multiple $x.A = y.B$ or $\mathcal{M}(x.\bar{A}, y.\bar{B})$, i.e., binary predicates at each leaf [21]. Hence so is for LBRs. $\square$

C. Proof of Proposition 3

Given a graph G , a set Σ of LBRs, and a set of ground truth Γ , we first show that all chasing sequences of G by (Σ, Γ) are finite. Consider a chasing sequence $\bar{T}_0, \bar{T}_1, \dots, \bar{T}_k$ of G by (Σ, Γ) . Observe that for each $i \in [0, k]$, $\bar{T}_i$ includes at most $O(|G|)$ many sets $[v.\text{label}]_{\supseteq}$ because the number of nodes v is bounded by $O(|G|)$. Furthermore, since each $[v.\text{label}]_{\supseteq}$ only contains attributes $v'.\text{label}$ and potential classification labels c , it has at most $O(|G| + B_u)$ many elements, where B_u denotes the upper bound for the number of class values that can be added to $[v.\text{label}]_{\supseteq}$.

Below we prove that such class values must appear in either Γ or Σ by induction on i , and therefore, $B_u = |\Gamma| + |\Sigma|$. Since $\bar{T}_0 = \Gamma$, obviously this holds for the base case where $i = 0$. Assume that for any $0 \leq j \leq i - 1$, all class values in the sets $[v.\text{label}]_{\supseteq}$ of $\bar{T}_j$ are from Γ or Σ . We show that the class values of new fixes added to $\bar{T}_i$ during chase step $\bar{T}_{i-1} \Rightarrow_{(\varphi_i, h_i)} \bar{T}_i$ also come from Γ or Σ . Based on the definition of the chase step, new class values can be included in $[v.\text{label}]_{\supseteq}$ in the following two cases: (a) $h_i(p_0)$ is $v.\text{label} = \{c\}$ or $c \in v.\text{label}$; or (b) $h_i(p_0)$ is $v.\text{label} = v'.\text{label}$ or $v'.\text{label} \subseteq v.\text{label}$, where p_0 is the consequence of LBR φ_i . For case (a), the class value c is added to $[v.\text{label}]_{\supseteq}$ and c is from Σ . In case (b), the class values are copied from $[v'.\text{label}]_{\supseteq}$ to $[v.\text{label}]_{\supseteq}$, which must appear in Γ or Σ since $[v'.\text{label}]_{\supseteq}$ is in $\bar{T}_{i-1}$. Note that there may also exist inclusion of class values when making all $[v.\text{label}]_{\supseteq}$ transitive, which must also be from Γ or Σ . One can verify this fact by using the same arguments for the above two cases. From these it follows that all class values in $\bar{T}_i$ are from Γ or Σ .

Thus, the total size of sets $[v.\text{label}]_{\supseteq}$ is at most $O((|G| +$

$|\Gamma| + |\Sigma|)^2$) for every $\bar{T}_i$ in the chasing sequence. Since each chase step expands the sets $[v.\text{label}]_{\supseteq}$ with at least one element and there are no elements removed, we can conclude that the chasing sequence must terminate in $O((|G| + |\Gamma| + |\Sigma|)^2)$ steps.

We next prove that all terminal chasing sequences of G by (Σ, Γ) converge at the same result by contradiction. Since the labels in graph G are updated regarding the fixes $\bar{T}_i$ but there is no topological change, it suffices to consider only whether all sequences converge at the same set of fixes.

Assume by contradiction that there exist two terminal chasing sequences of G by (Σ, Γ) that end up with different sets of fixes. We show that one of the chasing sequences is not terminal, i.e., the chasing sequence can be further expanded longer, a contradiction. Intuitively, given two chasing sequences ρ_1 and ρ_2 that yield different $\bar{T}_k$ and $\bar{T}'_l$, respectively, there exists a fix $r \in \bar{T}'_l \setminus \bar{T}_k$ (or $r \in \bar{T}_k \setminus \bar{T}'_l$) such that we can apply one more chase step to deduce r from $\bar{T}_k$ (or $\bar{T}'_l$), i.e., ρ_1 (or ρ_2) is not terminal.

More specifically, let $\rho_1 = \bar{T}_0 \Rightarrow_{(\varphi_1, h_1)} \bar{T}_1 \Rightarrow \dots \Rightarrow \bar{T}_{k-1} \Rightarrow_{(\varphi_k, h_k)} \bar{T}_k$ and $\rho_2 = \bar{T}'_0 \Rightarrow_{(\varphi'_1, h'_1)} \bar{T}'_1 \Rightarrow \dots \Rightarrow \bar{T}'_{l-1} \Rightarrow_{(\varphi'_l, h'_l)} \bar{T}'_l$ be two terminal chasing sequences of G by (Σ, Γ) that yield different fixes, i.e., $\bar{T}_k \neq \bar{T}'_l$. Assume w.l.o.g. that $\bar{T}'_l \setminus \bar{T}_k \neq \emptyset$. Let $\bar{T}_i$ and $\bar{T}'_i$ be the first pair of sets including different fixes in ρ_1 and ρ_2 , such that $\bar{T}'_i$ contains a fix that is not in $\bar{T}_i$, i.e., $\bar{T}'_{i-1} = \bar{T}_{i-1}$, $r \in \bar{T}'_i \setminus \bar{T}_i$ and $r \in \bar{T}'_l \setminus \bar{T}_k$. Such a pair must exist since ρ_1 and ρ_2 start with the same fixes, i.e., ground truth Γ ($\bar{T}_0 = \bar{T}'_0 = \Gamma$). Suppose that $\bar{T}'_{i-1} \Rightarrow_{(\varphi'_i, h'_i)} \bar{T}'_i$ is the step in ρ_2 that deduces this specific fix r . Below we prove that $\bar{T}_k \Rightarrow_{(\varphi'_i, h'_i)} \bar{T}_{k+1}$ is also a chase step, i.e., ρ_1 is not terminal, a contradiction. Here $\varphi'_i = \mathcal{Q}[x_0, y_0](X \rightarrow p_0)$ is an LBR in Σ , $\bar{T}_{k+1}$ expands $\bar{T}_k$ with the fix $h'_i(p_0) = r$ in the corresponding set $[v.\text{label}]_{\supseteq}$. Observe the following.

(1) We can also find h'_i as a match of pattern $\mathcal{Q}$ in graph G for sequence ρ_1 , since the two sequences still have the same fixes and graphs in their last steps and the topological structure of the graph G remains stable in the chasing sequences.

(2) Every predicate p in the precondition X of the LBR is validated by $\bar{T}_k$ at match h_i , because (a) p is validated by $\bar{T}'_{i-1}$ (by the assumption) using the sets $[v.\text{label}]_{\supseteq}$, e.g., $v'.\text{label} \subseteq v.\text{label}$ (resp. $v.\text{label} = v'.\text{label}$) is validated if $v'.$ label is covered in $[v.\text{label}]_{\supseteq}$ or there exists a value c in both $[v.\text{label}]_{\supseteq}$ and $[v'.$ label] $_{\supseteq}$, and moreover, (b) $\bar{T}'_{i-1} = \bar{T}_{i-1} \subseteq \bar{T}_k$.

Therefore, the step $\bar{T}_k \Rightarrow_{(\varphi'_i, h'_i)} \bar{T}_{k+1}$ can extend the chasing sequence ρ_1 , contradicting the assumption.

Note that the ML predicates in LBRs do not affect the Church-Rosser property when the embedded ML models are robust to graph attribute changes. Indeed, ML predicates can be treated in the same way as $x.A=y.B$, since the predictions of embedded models remain unchanged in response to attribute updates during the chase. Moreover, we compute probabilities of soft labels at the end of the chase based on the entire set of fixes derived and GNN model. Hence this does not affect the Church-Rosser property either. $\square$

D. Proof of Proposition 4

For the complexity of LCleaner, we first outline the algorithm LFix for deducing new fixes (LBR validation), which is invoked in each round of label imputation (line 5 of Algorithm 1). It is based on the notion of *scattered matches*.

Scattered matches. Consider a pattern Q_s in the form of a path $(u_0, l_0, u_1, \dots, u_{n-1}, l_{n-1}, u_n)$. The *scattered matches* of each u_i ($i \in [0, n]$) in graph G include all nodes v from G such that (a) $T(v) = T_{Q_s}(u_i)$, and (b) when $i < n$, there exists an edge (v, l_i, v') in G with v' being a scattered match of u_{i+1} . Similarly, we define the *bidirectional scattered matches*. For each u_i in Q_s , the *bidirectional scattered matches* of u_i contain nodes v such that (a) $T(v) = T_{Q_s}(u_i)$, and (b) if u_i has a child (resp. parent) in Q_s , then there exists an edge (v, l_i, v') (resp. (v', l_{i-1}, v)) in G that has v' as a bidirectional scattered match of u_{i+1} (resp. u_{i-1}).

Intuitively, every standard (complete) match in pattern matching via homomorphism maps the entire set of pattern nodes from Q_s as a whole (Section II). In contrast, the scattered matches are defined over distinct pattern nodes from the path Q_s , while they recursively enforce (partial) requirements on edge connections as in the standard pattern matching. Obviously this can reduce the upper bound on the number of matches from $O(|G|^{|Q_s|})$ (complete matches) to $O(|Q_s||G|)$ (scattered matches) since each pattern node has at most $|G|$ scattered matches. Moreover, by induction on i and comparing the semantics of complete matches and scattered matches, one can verify that the bidirectional scattered matches of every u_i from Q_s must appear in the complete matches of Q_s . Indeed, we can assemble the bidirectional scattered matches of different pattern nodes to restore complete matches. In addition, the scattered matches of the first node u_0 on path must be involved in the complete matches of Q_s , while the rest of scattered matches are potentially contained in complete matches.

Deducing fixes. Employing scattered matches, algorithm LFix works in four phases. Given a set Σ of LBRs, a graph G and the fixes $\bar{T}_k$, it (1) decomposes each dual pattern $\mathcal{Q}$ from Σ into a set of paths w.r.t. the binary predicates in preconditions X of LBRs; (2) identifies the scattered matches of the leaves in G that satisfy X regarding the fixes $\bar{T}_k$, i.e., all predicates in X are validated by $\bar{T}_k$, and maps other pattern nodes from $\mathcal{Q}$ in a bottom-up manner along pairs of decomposed paths using dynamic programming [21], to derive scattered matches for internal pattern nodes; (3) decides the scattered matches of the centers of $\mathcal{Q}$ by aggregating the results, i.e., computing the intersection of those scattered matches for centers that are obtained along different decomposed paths; and (4) deduces the fixes induced by these pairs of scattered matches w.r.t. the consequences p_0 of LBRs, which essentially computes the bidirectional scattered matches in a top-down manner with dynamic programming [21] again.

By the properties of (bidirectional) scattered matches, the scattered matches of the centers established in phase (3) must be involved in the complete matches that satisfy preconditions X ; and the fixes deduced in phase (4) with the top-down approach must also appear in the complete matches. These

guarantee the correctness of algorithm LFix. Note that LFix processes X and p_0 differently, since binary predicates in X are defined only on two leaves or centers from two disjoint stars of $\mathcal{Q}$, while p_0 can be defined arbitrarily.

Note that when validating the set Σ of LBRs over G , the input $\bar{T}_k$ covers the entire G , and algorithm LFix checks whether the scattered matches obtained in phase (3) induce complete matches that satisfy the consequences p_0 during phase (4).

For its complexity, decomposing dual patterns *w.r.t.* binary predicates takes $O(|\Sigma|)$ time in phase (1), and computing scattered matches of leaves that satisfy preconditions X needs $O(|\Sigma||G|^2 C_m)$ time in phase (2), **where C_m is the cost for validating an ML predicate, a polynomial in $|G|$ for most common ML models [21], [24], [25]**. The latter covers the costs for checking set predicates and 1-WL predicates since the graded bisimilarity relation can be derived in advance in $O(|G| \log |G|)$ time, and all potential classification labels are chosen from a fixed set. In addition, LFix takes $(|\Sigma||G|^3)$ time to identify scattered matches for all other nodes from the dual patterns in the bottom-up computation [86] during phase (2). Determining the scattered matches of the centers takes $O(|\Sigma||G|^2)$ time in phase (3), because there are at most $O(|G|^2)$ many scattered matches for the two centers of each dual pattern derived along each pair of decomposed paths. Finally, the top-down procedure in phase (4) also takes $(|\Sigma||G|^3)$ time. Putting these together, LFix runs in $O(C_m|\Sigma||G|^3)$ time.

Complexity. We now study the complexity of LCleaner. Observe that by pairwise comparing the LBRs in Σ and the new fixes deduced, selection of affected LBRs in each iteration can be done in $O(|\Sigma||G|^2)$ time (lines 3–4 of Algorithm 1). Since there are at most $O((|G| + |\Gamma| + |\Sigma|)^2)$ iterations, *i.e.*, the maximum length of chasing sequences (proof of Proposition 3), and deducing fixes needs $O(C_m|\Sigma||G|^3)$ time as shown above, the iterative process of LCleaner takes $O(C_m|\Sigma||G|^3(|G| + |\Gamma| + |\Sigma|)^2)$ time (lines 2–6 of Algorithm 1). Computing the probabilities of soft labels is in $O(|G|(|\Gamma| + |\Sigma|))$ time as each node has at most $O(|\Gamma| + |\Sigma|)$ potential class values (line 7 of Algorithm 1). Taken together, LCleaner takes $O(C_m|\Sigma||G|^3(|G| + |\Gamma| + |\Sigma|)^2)$ time. $\square$

E. Parallel Algorithm

To scale with large graphs, we parallelize LCleaner (Section V) and show that it guarantees the parallel scalability [68].

Parallel Scalability. We adapt the notion of [68] to characterize the effectiveness of parallel algorithms. Denote by $t_s(|I|)$ the worst-case cost of a sequential algorithm $\mathcal{A}$ related to LBRs, where $|I|$ denotes the size of the input instance for $\mathcal{A}$. We say that a parallel algorithm $\mathcal{A}_p$ is *parallelly scalable relative to $\mathcal{A}$* if $\mathcal{A}_p$ and $\mathcal{A}$ output the same result *w.r.t.* input instance I and when using n processors, its parallel cost is

$$t_p(|I|, n) = O\left(\frac{t_s(|I|)}{n}\right).$$

The parallel scalability measures the speedup over a yardstick sequential algorithm $\mathcal{A}$ by parallelization. A parallelly scalable $\mathcal{A}_p$ “linearly” reduces the cost of $\mathcal{A}$ when n increases.

Parallelization. Observe that the sequential LCleaner iteratively computes the fixes using Σ and Γ . In each iteration, it (a) selects an appropriate set Σ_d of LBRs that can be enforced to deduce new fixes, and (b) validates the set of selected LBRs in graph G . These two steps take $O(|\Sigma||G|^2)$ time and $O(C_m|\Sigma||G|^3)$ time, respectively. Here C_m refers to the cost for checking a pre-trained ML predicate. Moreover, the cost of computing probabilities for soft labels is dominated by that of fix deduction, *i.e.*, LBR validation.

To this end, we propose a parallel algorithm by parallelizing LCleaner, denoted as PLCleaner; it applies a parallel LBR validation strategy and performs sequential LBR selection and soft label computation. The strategy validates LBRs in $O(C_m|\Sigma||G|^3/n)$ time with n processors.

More specifically, recall that the sequential LBR validation algorithm has four main steps (see the proof of Proposition 4): (1) decompose the patterns into paths, which is done by a linear scan of the dual patterns in Σ , taking $O(|\Sigma|)$ time; (2) execute the workloads constructed *w.r.t.* the decomposed pattern paths in a bottom-up way, in $O(|\Sigma||G|^2 C_m)$ time; (3) conduct matching result aggregation at the centers of the patterns in LBRs, in $O(|\Sigma||G|^2)$ time; and (4) perform consequence verification with the top-down strategy, in $O(|\Sigma||G|^3)$ time. Moreover, steps (2) and (4) are triggered by (a) *independent* pairs of pattern paths (and their leaves) and (b) aggregated match pairs of the centers, respectively. Thus, we do the tasks regarding different triggers in parallel for the two steps. Using the bulk synchronous parallel (BSP) model [164], we parallelize LBR validation in three supersteps as follows.

(1) Decompose the patterns in the LBRs of Σ into workloads at a designated coordinator (processor) as in the sequential LCleaner. Then the workloads (pairs of pattern paths) are evenly distributed to all processors; and the relevant part of the graph and sets $[v.\text{label}]_{\geq}$ are also transmitted to corresponding processors for pattern matching. At last, the processors execute the received workloads with the bottom-up strategy in parallel and send results back to the coordinator.

(2) In the next superstep, matching result aggregation is performed locally at the coordinator, and the aggregated match pairs are evenly partitioned and distributed to different processors. In addition, we shuffle useful candidate scattered matches computed in the last superstep to all processors, *i.e.*, those to be inspected for checking consequences and deducing fixes, such that the validation can also be conducted in parallel.

(3) In the last superstep, all processors validate the consequences of the LBRs in parallel with the received match pairs.

Analysis. This parallel process takes $O(C_m|\Sigma||G|^3/n)$ time, because (a) parallel workload execution and consequence verification take $O(C_m|\Sigma||G|^3/n)$ and $O(|\Sigma||G|^3/n)$ time, respectively, due to the even partition of the triggers; and (b) data shuffling takes $O(|\Sigma||G|^2)$ time as there exist at most $O(|\Sigma||G|^2)$ candidate scattered matches. Since there are at most $O((|G| + |\Gamma| + |\Sigma|)^2)$ iterations, PLCleaner takes $O(C_m|\Sigma||G|^3(|G| + |\Gamma| + |\Sigma|)^2/n)$ time in total. Therefore, it is parallelly scalable relative to LCleaner.